1. Project Profile

Project Title	**Enhancing Cybersecurity with Machine Learning: Leveraging UNSW-NB15 Dataset**
Objective	To develop multiple machine learning models including Random Forest, LightGBM, Decision Trees, and Linear Regression to identify and classify different types of network attacks.
Platform	Visual Studio Code 1.89.1
Tool	Python 3.11.3 , Matplotlib, scikit-learn, lightgbm, pandas, joblib, numpy
Internal Guide	Dr. Kashyap C. Patel
Prepared By	Devarshi Patel (21082291044) & Yash Chauhan (21082291037)

1.1 Objective

To develop multiple machine learning models including Random Forest, LightGBM, Decision Trees, and Linear Regression to identify and classify different types of network attacks.

To compare the performance of these models in terms of accuracy, precision, recall, F1-score, and ROC-AUC to determine the most effective approach for cyber threat detection.

To design and simulate a real-time intrusion detection system using the best-performing model.

2. Requirement Specification

2.1 Existing Systems

- The current cybersecurity landscape within the organization relies primarily on traditional signature-based detection systems and rule-based security measures.
- These systems scan for known patterns of malicious activity in network traffic and endpoint processes.
- The reliance on predefined rules can result in frequent false alarms.
- The lack of predictive analytics and machine learning integration means the system is not utilizing the full scope of available data to anticipate potential threats.

2.2 Need For New System

- Utilize machine learning models capable of learning from ongoing traffic and adapting to new, unseen cyber threats.
- Support continuous learning and model retraining with new data to improve accuracy over time.
- Integrate automated response mechanisms that can execute predefined actions on detected threats to mitigate damage without manual intervention.

3. Minimum Hardware Requirements

3.1 Hardware Requirement

Server Configuration :-

Processor	:	Core i3 or Higher
RAM	:	8 GB
GPU	:	4 GB
SSD	:	128 GB

Client Configuration :-

Processor	:	Core i3 or Higher
RAM	:	8 GB
Hard Disk/SSD	:	128 GB

4. Algorithms and Methods

1. **Pandas**: Pandas is a powerful open-source Python library for data manipulation and analysis. It provides data structures and data analysis tools for working with structured (tabular, multidimensional, potentially heterogeneous) and time series data.

2. **scikit-learn**: Scikit-learn is a popular machine learning library for Python. It features various classification, regression, and clustering algorithms, as well as tools for model evaluation and data preprocessing.

    - train_test_split: A utility function from scikit-learn that splits the dataset into training and testing subsets.
    - LabelEncoder: A utility class for encoding categorical variables as numeric values.
    - RandomForestClassifier: An ensemble learning algorithm that builds multiple decision trees and combines their predictions.
    - LogisticRegression: A statistical model for binary classification problems.
    - DecisionTreeClassifier: A decision tree algorithm for classification tasks.

3. **LightGBM**: LightGBM (Light Gradient Boosting Machine) is a gradient boosting framework that uses tree-based learning algorithms. It is designed to be highly efficient and can handle large-scale data and high-dimensional features.

    Scikit-learn Metrics:

    - accuracy_score: Computes the accuracy score, which is the fraction of correctly classified instances.
    - f1_score: Calculates the F1 score, which is the harmonic mean of precision and recall.
    - confusion_matrix: Computes the confusion matrix, which provides a breakdown of correct and incorrect classifications.
    - roc_auc_score: Computes the area under the receiver operating characteristic (ROC) curve for binary classification problems.
    - roc_curve: Computes the receiver operating characteristic (ROC) curve, which plots the true positive rate against the false positive rate.

- precision_recall_curve: Computes the precision-recall curve, which plots precision against recall for different probability thresholds.

4. **Matplotlib**: Matplotlib is a plotting library for Python, which produces publication-quality figures in a variety of hardcopy formats and interactive environments across platforms.

5. **Joblib**: Joblib is a set of tools to provide lightweight pipelining in Python. It can be used to persist and reload Python objects that can be reused across various tasks.

6. **NumPy**: NumPy is a fundamental library for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of high-level mathematical functions to operate on these arrays.

```
# Sample Example of Code Used to Train models
models = {
"Linear Regression": LogisticRegression(),
"Random Forest": RandomForestClassifier(),
"LightGBM": LGBMClassifier(),
"Decision Trees": DecisionTreeClassifier()
}
```

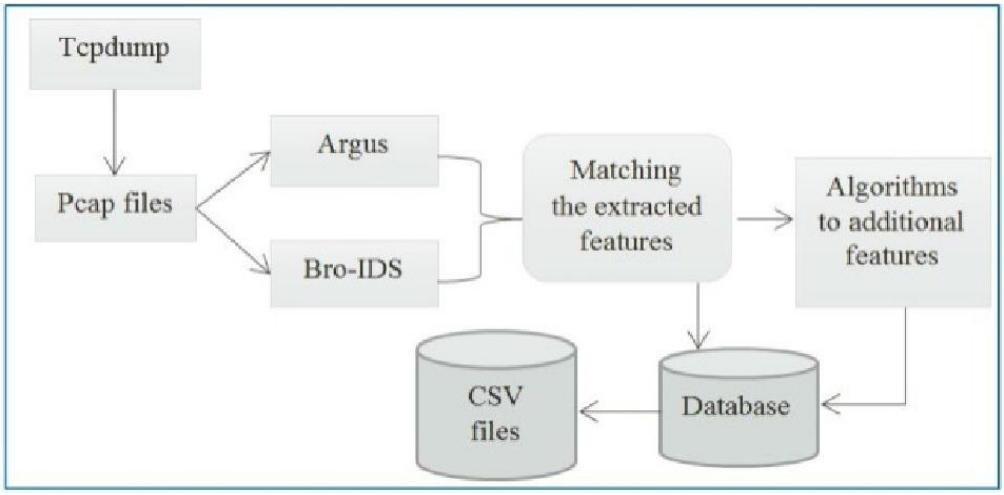
5. Dataset

UNSW_NB15

The UNSW-NB 15 dataset is a comprehensive network intrusion detection dataset created by the Australian Centre for Cyber Security (ACCS). It was generated using the IXIA PerfectStorm tool, which simulated a hybrid of real modern normal activities and synthetic contemporary attack behaviors. The dataset contains 100 GB of raw network traffic captured using the Tcpdump tool, comprising nine types of attacks, including Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. The dataset features 49 different features, which are described in the UNSW-NB15_features.csv file.

The UNSW-NB 15 dataset consists of over two million records, divided into four CSV files: UNSW-NB15_1.csv, UNSW-NB15_2.csv, UNSW-NB15_3.csv, and UNSW-NB15_4.csv. It also includes a ground truth table (UNSW-NB15_GT.csv) and a list of events (UNSW-NB15_LIST_EVENTS.csv). The dataset is partitioned into a training set (UNSW_NB15_training-set.csv) with 175,341 records and a testing set (UNSW_NB15_testing-set.csv) with 82,332 records, containing different types of attacks and normal traffic. The dataset's creation and its features are described in several published papers by the authors.

The authors encourage researchers to use and cite the new techniques and datasets developed for intrusion detection, network forensics, privacy-preserving, and threat intelligence applications when comparing their work with the UNSW-NB15 dataset. They have published several papers on these topics, including ensemble intrusion detection techniques, network forensic mechanisms for botnet activities, Web application attack detection, privacy preservation for SCADA systems, threat intelligence schemes for Industry 4.0, and anomaly detection using beta mixture models. The authors have granted free use of the UNSW-NB15 dataset for academic research purposes but require citation of their published works for commercial use.



DATA SET RECORD DISTRIBUTION

Type	No. Records	Description
Normal	2,218,761	Natural transaction data.
Fuzzers	24,246	Attempting to cause a program or network suspended by feeding it the randomly generated data.
Analysis	2,677	It contains different attacks of port scan, spam and html files penetrations.
Backdoors	2,329	A technique in which a system security mechanism is bypassed stealthily to access a computer or its data.
DoS	16,353	A malicious attempt to make a server or a network resource unavailable to users, usually by temporarily interrupting or suspending the services of a host connected to the Internet.
Exploits	44,525	The attacker knows of a security problem within an operating system or a piece of software and leverages that knowledge by exploiting the vulnerability.
Generic	215,481	A technique works against all blockciphers (with a given block and key size), without consideration about the structure of the block-cipher.
Reconnaissance	13,987	Contains all Strikes that can simulate attacks that gather information.

Shellcode	1,511	A small piece of code used as the payload in the exploitation of software vulnerability.
Worms	174	Attacker replicates itself in order to spread to other computers. Often, it uses a computer network to spread itself, relying on security failures on the target computer to access it.

DATA SET FEATURES

No.	Name	Type	Description
1	srcip	nominal	Source IP address
2	sport	integer	Source port number
3	dstip	nominal	Destination IP address
4	dsport	integer	Destination port number
5	proto	nominal	Transaction protocol
6	state	nominal	Indicates to the state and its dependent protocol, e.g. ACC, CLO, CON, ECO, ECR, FIN, INT, MAS, PAR, REQ, RST, TST, TXD, URH, URN, and (-) (if not used state)
7	dur	Float	Record total duration
8	sbytes	Integer	Source to destination transaction bytes
9	dbytes	Integer	Destination to source transaction bytes
10	sttl	Integer	Source to destination time to live value
11	dttl	Integer	Destination to source time to live value
12	sloss	Integer	Source packets retransmitted or dropped
13	dloss	Integer	Destination packets retransmitted or dropped
14	service	nominal	http, ftp, smtp, ssh, dns, ftp-data ,irc and (-) if not much used service
15	Sload	Float	Source bits per second
16	Dload	Float	Destination bits per second
17	Spkts	integer	Source to destination packet count
18	Dpkts	integer	Destination to source packet count
19	swin	integer	Source TCP window advertisement value
20	dwin	integer	Destination TCP window advertisement value
21	stcpb	integer	Source TCP base sequence number
22	dtcpb	integer	Destination TCP base sequence number
23	smeansz	integer	Mean of the ?ow packet size transmitted by the src
24	dmeansz	integer	Mean of the ?ow packet size transmitted by the dst
25	trans_depth	integer	Represents the pipelined depth into the connection of http request/response transaction
26	res_bdy_len	integer	Actual uncompressed content size of the data transferred from the server's http service.
27	Sjit	Float	Source jitter (mSec)
28	Djit	Float	Destination jitter (mSec)
29	Stime	Timestamp	record start time
30	Ltime	Timestamp	record last time

31	Sintpkt	Float	Source interpacket arrival time (mSec)
32	Dintpkt	Float	Destination interpacket arrival time (mSec)
33	tcprtt	Float	TCP connection setup round-trip time, the sum of 'synack' and 'ackdat'.
34	synack	Float	TCP connection setup time, the time between the SYN and the SYN_ACK packets.
35	ackdat	Float	TCP connection setup time, the time between the SYN_ACK and the ACK packets.
36	is_sm_ips_ports	Binary	If source (1) and destination (3)IP addresses equal and port numbers (2)(4) equal then, this variable takes value 1 else 0
37	ct_state_ttl	Integer	No. for each state (6) according to specific range of values for source/destination time to live (10) (11).
38	ct_flw_http_mthd	Integer	No. of flows that has methods such as Get and Post in http service.
39	is_ftp_login	Binary	If the ftp session is accessed by user and password then 1 else 0.
40	ct_ftp_cmd	integer	No of flows that has a command in ftp session.
41	ct_srv_src	integer	No. of connections that contain the same service (14) and source address (1) in 100 connections according to the last time (26).
42	ct_srv_dst	integer	No. of connections that contain the same service (14) and destination address (3) in 100 connections according to the last time (26).
43	ct_dst_ltm	integer	No. of connections of the same destination address (3) in 100 connections according to the last time (26).
44	ct_src_ ltm	integer	No. of connections of the same source address (1) in 100 connections according to the last time (26).
45	ct_src_dport_ltm	integer	No of connections of the same source address (1) and the destination port (4) in 100 connections according to the last time (26).
46	ct_dst_sport_ltm	integer	No of connections of the same destination address (3) and the source port (2) in 100 connections according to the last time (26).
47	ct_dst_src_ltm	integer	No of connections of the same source (1) and the destination (3) address in in 100 connections according to the last time (26).
48	attack_cat	nominal	The name of each attack category. In this data set , nine categories e.g. Fuzzers, Analysis, Backdoors, DoS Exploits, Generic, Reconnaissance, Shellcode and Worms
49	Label	binary	0 for normal and 1 for attack records

5.1 Dataset code step:

Step 1: Loading the Datasets

```
train_dataset = pd.read_csv("D:\\Project\\cyber\\GG/UNSW_NB15_training-set.csv")

test_dataset = pd.read_csv("D:\\Project\\cyber\\GG/UNSW_NB15_testing-set.csv")
```

In this step, the training and testing datasets are loaded from the respective CSV files using the pd.read_csv function from the Pandas library.

Step 2: Preprocessing

```
label_encoder = LabelEncoder()
train_dataset['label'] = label_encoder.fit_transform(train_dataset['attack_cat'])
test_dataset['label'] = label_encoder.transform(test_dataset['attack_cat'])
```

In the preprocessing step, the LabelEncoder class from scikit-learn is used to convert the categorical attack categories (attack_cat) into numerical labels. The fit_transform method is applied on the training dataset to fit the encoder and transform the categories into labels. For the testing dataset, only the transform method is used to ensure consistency with the same label encoding as the training dataset.

```
attack_categories = dict(enumerate(label_encoder.classes_))
```

This line creates a dictionary mapping the numerical labels to their corresponding attack category names. This mapping will be useful for interpreting the results later.

Step 3: Feature Selection

```
features = ['id', 'dur', 'spkts', 'dpkts', 'sbytes', 'dbytes', 'rate', 'sttl', 'dttl', 'sload', 'dload', 'sloss', 'dloss', 'sinpkt', 'dinpkt', 'sjit', 'djit', 'swin', 'stcpb', 'dtcpb', 'dwin', 'tcprtt', 'synack', 'ackdat', 'smean', 'dmean', 'trans_depth', 'response_body_len', 'ct_srv_src', 'ct_state_ttl', 'ct_dst_ltm', 'ct_src_dport_ltm', 'ct_dst_sport_ltm', 'ct_dst_src_ltm', 'is_ftp_login', 'ct_ftp_cmd', 'ct_flw_http_mthd', 'ct_src_ltm', 'ct_srv_dst', 'is_sm_ips_ports']
```

This line defines a list of feature names that will be used for training and testing the machine learning models. These features are selected based on their relevance to the network intrusion detection task.

Step 4: Dataset Splitting

```
X_train = train_dataset[features]

y_train = train_dataset['label']

X_test = test_dataset[features]

y_test = test_dataset['label']
```

In this step, the training and testing datasets are split into features (X_train, X_test) and labels (y_train, y_test). The features are selected based on the list defined in the previous step, and the labels correspond to the numerical labels obtained from the LabelEncoder.

After these steps, the data is ready for training machine learning models and evaluating their performance on the testing dataset. The subsequent steps would involve:

1. Instantiating and training different machine learning models (e.g., Random Forest, LightGBM, Logistic Regression, Decision Tree) using the training data (X_train, y_train).
2. Evaluating the trained models on the testing data (X_test, y_test) using appropriate evaluation metrics (e.g., accuracy, F1-score, confusion matrix, ROC-AUC score).
3. Analyzing the results and selecting the best-performing model(s) for the network intrusion detection task.
4. Optionally, tuning the hyperparameters of the selected model(s) to improve their performance further.
5. Interpreting the results and providing insights into the model's predictions and the importance of various features.

5.2 Performance Metrics:

1. **Accuracy:**

   Accuracy is the ratio of correctly predicted instances to the total number of instances. It provides a straightforward measure of the overall performance of a

classification model. However, accuracy can be misleading when dealing with imbalanced datasets, as it does not account for the distribution of classes.

$$Accuracy = \frac{Number\ of\ Correct\ Predictions}{Total\ Number\ of\ Predictions}$$

2. **Recall:**

Recall, also known as sensitivity or true positive rate, measures the proportion of actual positive instances that are correctly identified by the model. It quantifies the model's ability to detect all relevant instances and is particularly important in applications where missing positive instances is costly, such as disease diagnosis or fraud detection. A high recall indicates that the model is effective in capturing most of the positive cases, but it may also lead to a higher number of false positives.

$$Recall = \frac{TP}{TP + FN}$$

		Real Class	
		Positive	**Negative**
Predicted Class	**Positive**	TP	FP
	Negative	FN	TN

3. **Precision:**

Precision measures the proportion of instances predicted as positive that are actually positive. It quantifies the model's ability to avoid false alarms or false positives, which is crucial in scenarios where false positives are costly or undesirable, such as spam filtering or intrusion detection systems. A high precision indicates that the model makes fewer incorrect positive predictions, but it may also result in missing some true positive cases. Precision is particularly

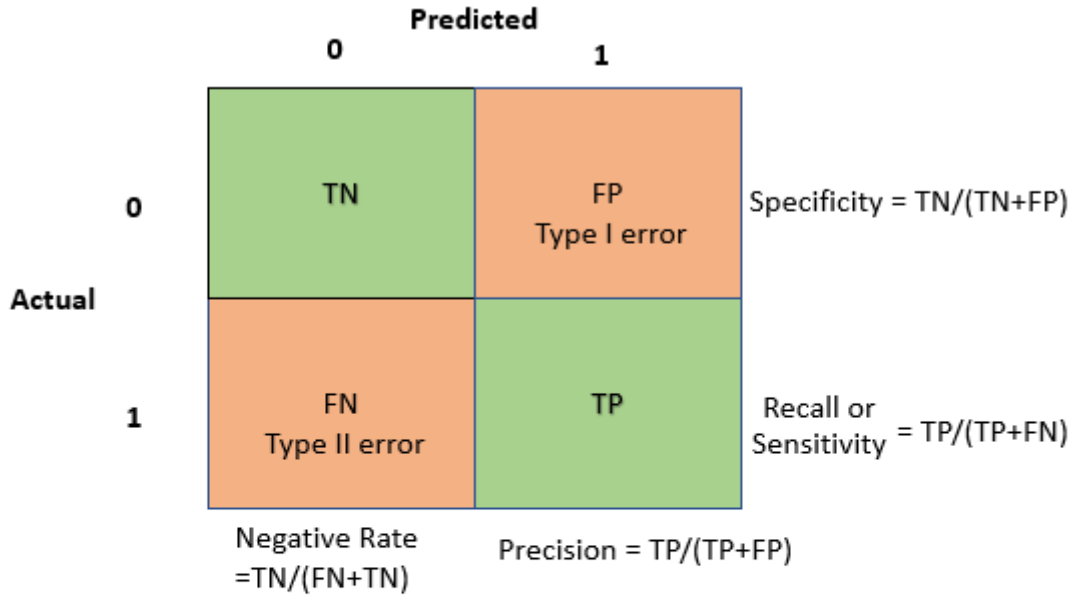
relevant when the cost of a false positive is high or when resources are limited for investigating potential positive instances.

$$Precision = \frac{TP}{TP + FP}$$

		Real Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN

4. F1-score:

The F1-score is the harmonic mean of precision (the fraction of true positives among the instances predicted as positive) and recall (the fraction of true positives among the actual positive instances). It provides a balanced measure of a model's performance, especially when dealing with imbalanced classes or unequal costs of false positives and false negatives.

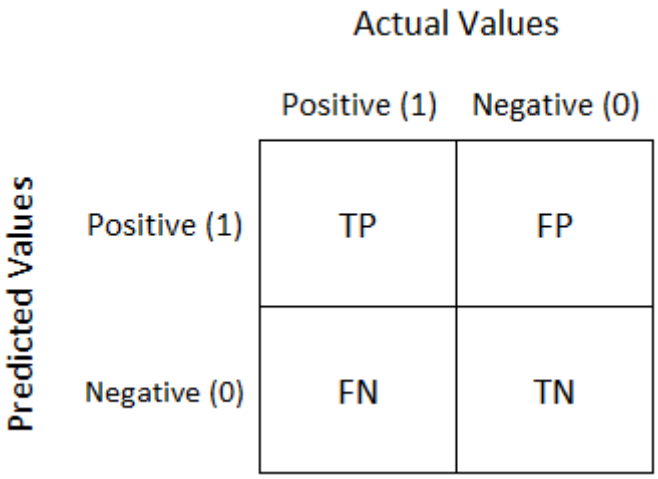


$$\text{Accuracy} = \frac{TP+TN}{TP+FP+TN+FN}$$

$$\text{F1 - Score} = \frac{2*\text{Recall}*\text{Precision}}{\text{Recall} + \text{Precision}}$$

5. Confusion Matrix:

The confusion matrix is a tabular representation that summarizes the performance of a classification model by showing the counts of true positives, true negatives, false positives, and false negatives. It provides a detailed breakdown of the model's predictions and serves as a basis for computing various performance metrics.



6. ROC-AUC Score:

The Receiver Operating Characteristic (ROC) Area Under the Curve (AUC) score is a measure of the model's ability to distinguish between classes. It represents the probability that a randomly chosen positive instance will be ranked higher than a randomly chosen negative instance. The ROC-AUC score ranges from 0 to 1, with higher values indicating better performance.

```
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average='macro')
conf_matrix = confusion_matrix(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_prob, multi_class='ovr')
# Compute ROC curve and Precision-Recall curve
n_classes = len(np.unique(y_test))
```

7. ROC Curve:

The ROC (Receiver Operating Characteristic) curve is a graphical representation of the trade-off between the true positive rate (sensitivity) and the false positive rate (1 - specificity) at different classification thresholds. It provides a visual assessment of the model's performance and helps in selecting an appropriate threshold based on the desired balance between true positives and false positives.

```
plt.figure()
for i in range(n_classes):
fpr, tpr, _ = roc_curve(y_test == i, y_prob[:, i])
plt.plot(fpr, tpr, label=f"{attack_categories[i]} (AUC =
{roc_auc_score(y_test == i, y_prob[:, i]):.2f})")
plt.plot([0, 1], [0, 1], 'k--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f"{name} ROC Curve (Multiclass)")
plt.legend(loc="lower right")
plt.show()
```

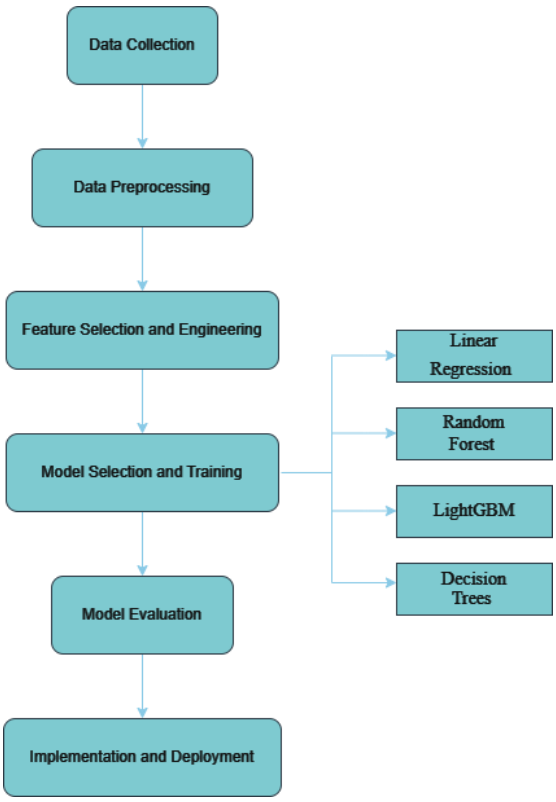
8. Precision-Recall Curve:

The Precision-Recall curve plots precision (the fraction of true positives among the instances predicted as positive) against recall (the fraction of true positives among the actual positive instances) at different classification thresholds. It is particularly useful when evaluating models on imbalanced datasets, as it focuses on the performance of the positive class rather than the overall accuracy.

```
plt.figure()
for i in range(n_classes):
precision, recall, _ = precision_recall_curve(y_test == i, y_prob[:, i])
plt.plot(recall, precision, label=f"{attack_categories[i]}")
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title(f"{name} Precision-Recall Curve (Multiclass)")
plt.legend(loc="lower left")
plt.show()
```

6. PROCESS FLOW

- Data Collection: This is the initial step where raw data is gathered from various sources.
- Data Preprocessing: The collected data goes through preprocessing steps, which may include cleaning, formatting, and transforming the data into a suitable format for further analysis.
- Feature Selection and Engineering: In this step, relevant features or variables are selected and engineered from the preprocessed data. This process may involve techniques like dimensionality reduction, feature extraction, or feature creation.
- Model Selection and Training: Different machine learning models are selected and trained on the processed data. The image shows four example models: Linear Regression, Random Forest, LightGBM, and Decision Trees.
- Model Evaluation: The trained models are evaluated using various performance metrics to assess their accuracy, precision, recall, or other relevant measures.
- Implementation and Deployment: The best-performing model(s) are selected and deployed into a production environment for real-world use or decision-making.



7. RESULTS

Before Hyper Parameter

Linear Regression Accuracy: 0.6100686220926702

Random Forest Accuracy: 0.9001639642922208

LightGBM Accuracy: 0.8713183943644865

Decision Trees Accuracy: 0.8889293738993138

Linear Regression Accuracy: 0.6104329871864942 (--)

Random Forest Accuracy: 0.9001032367765834 (--)

LightGBM Accuracy: 0.8816420720228335 (++)

Decision Trees Accuracy: 0.8416226392178296 (++)

"Linear Regression": LogisticRegression(max_iter=100),

"Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),

"LightGBM": LGBMClassifier(n_estimators=100, learning_rate=0.1, max_depth=5),

"Decision Trees": DecisionTreeClassifier(max_depth=5)

After Hyper parameter Exp-1

Linear Regression Accuracy: 0.5507378393149936 (--)

Random Forest Accuracy: 0.9006497844173195 (++)

LightGBM Accuracy: 0.6051496933260461 (--)

Decision Trees Accuracy: 0.8973704985729034 (++)

"Linear Regression": LogisticRegression(max_iter=10),

"Random Forest": RandomForestClassifier(n_estimators=500, random_state=82),

"LightGBM": LGBMClassifier(n_estimators=1000, learning_rate=0.2, max_depth=10),

"Decision Trees": DecisionTreeClassifier(max_depth=10)

After Hyper parameter Exp-2

Linear Regression Accuracy: 0.6299265197060788 (++)

Random Forest Accuracy: 0.9007712394485942 (++)

LightGBM Accuracy: 0.5851703406813628 (--)

Decision Trees Accuracy: 0.8972490435416287 (--)

"Linear Regression": LogisticRegression(max_iter=10000),

"Random Forest": RandomForestClassifier(n_estimators=1000, random_state=256),

"LightGBM": LGBMClassifier(n_estimators=1000, learning_rate=0.1, max_depth=-1, num_leaves=31,),

"Decision Trees": DecisionTreeClassifier(max_depth=10)

After Hyper Parameter Exp-3

Linear Regression Accuracy: 0.6101293496083076 (--)

Random Forest Accuracy: 0.9008926944798689 (++)

LightGBM Accuracy: 0.8795773364911641 (--)

Decision Trees Accuracy: 0.8871075484301937 (--)

Last Testing Exp-4

Linear Regression Accuracy: 0.6100686220926702

Random Forest Accuracy: 0.9008926944798689

LightGBM Accuracy: 0.8713183943644865

Decision Trees Accuracy: 0.8885650088054897

"Linear Regression": LogisticRegression(max_iter=1000),

"Random Forest": RandomForestClassifier(n_estimators=1000, random_state=42),

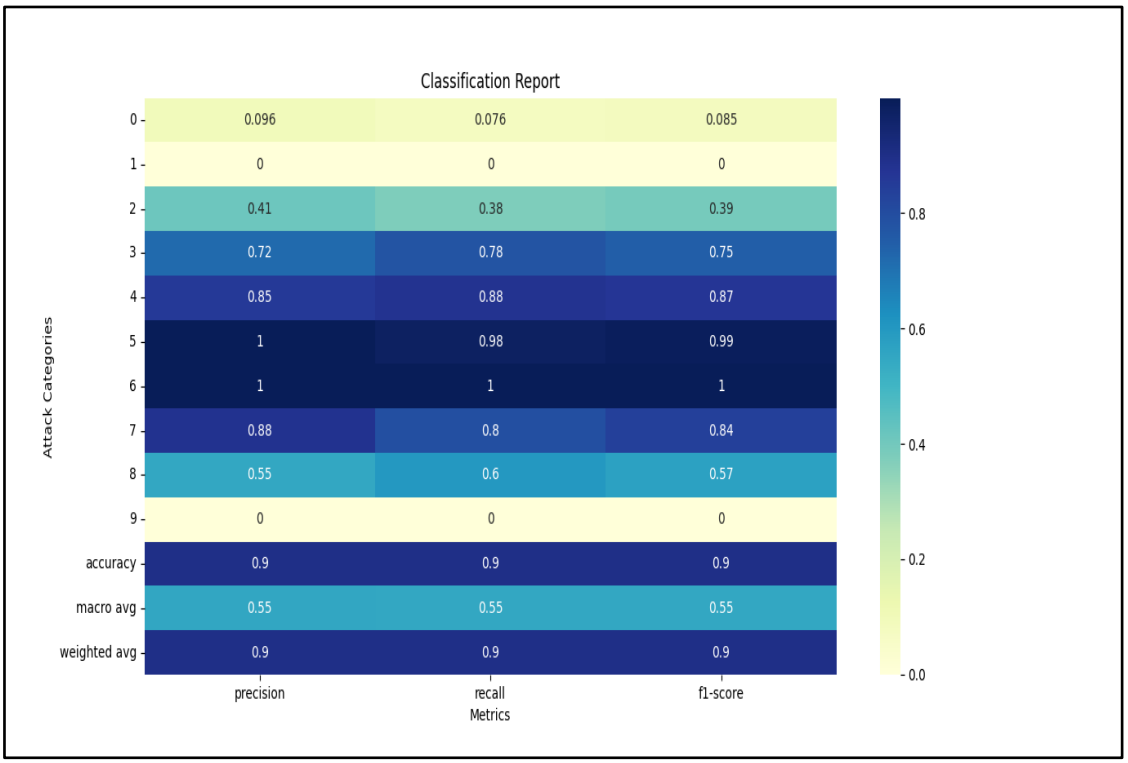
"LightGBM": LGBMClassifier(num_leaves=31, max_depth=-1),

"Decision Trees": DecisionTreeClassifier()

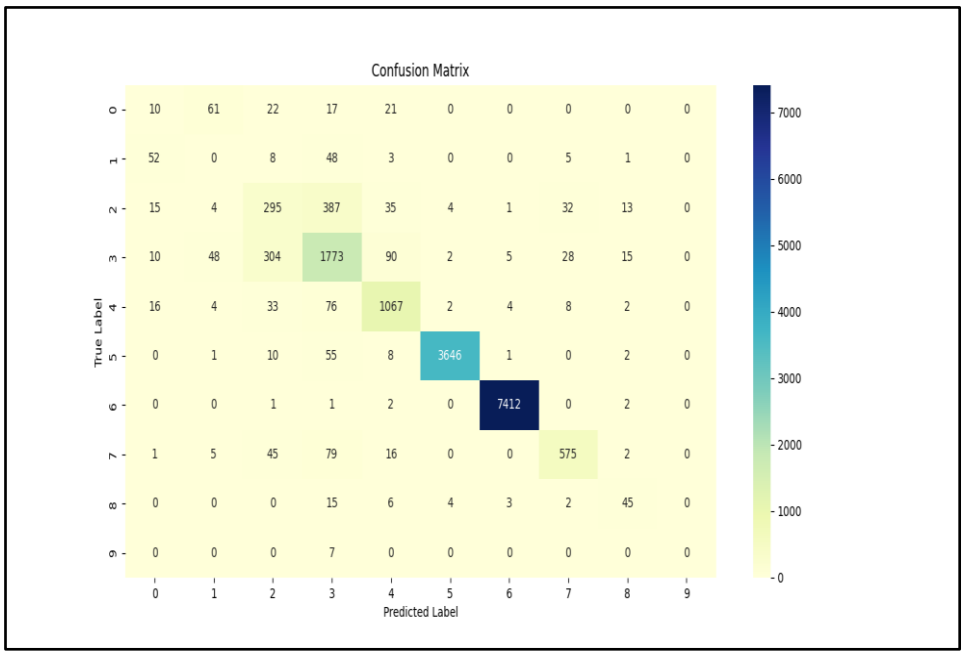
6.1 Evaluation Metrics

Evaluation metrics are crucial in assessing the effectiveness and performance of machine learning models. In our project, we employed various evaluation metrics to measure the accuracy and reliability of our models in classifying cybersecurity threats. These metrics include accuracy, precision, recall, and F1-score. Accuracy represents the overall correctness of the model's predictions, indicating the proportion of correctly classified instances among all instances. Precision measures the proportion of true positive predictions among all positive predictions made by the model, focusing on the model's ability to avoid false positives. Recall, on the other hand, evaluates the model's ability to capture all positive instances by measuring the proportion of true positive predictions among all actual positive instances in the dataset. F1-score combines precision and recall into a single metric, providing a balanced assessment of a model's performance by considering both false positives and false negatives. By meticulously evaluating our models using these metrics, we gained valuable insights into their strengths and weaknesses, enabling us to make informed decisions and select the most suitable approach for enhancing cybersecurity with machine learning techniques

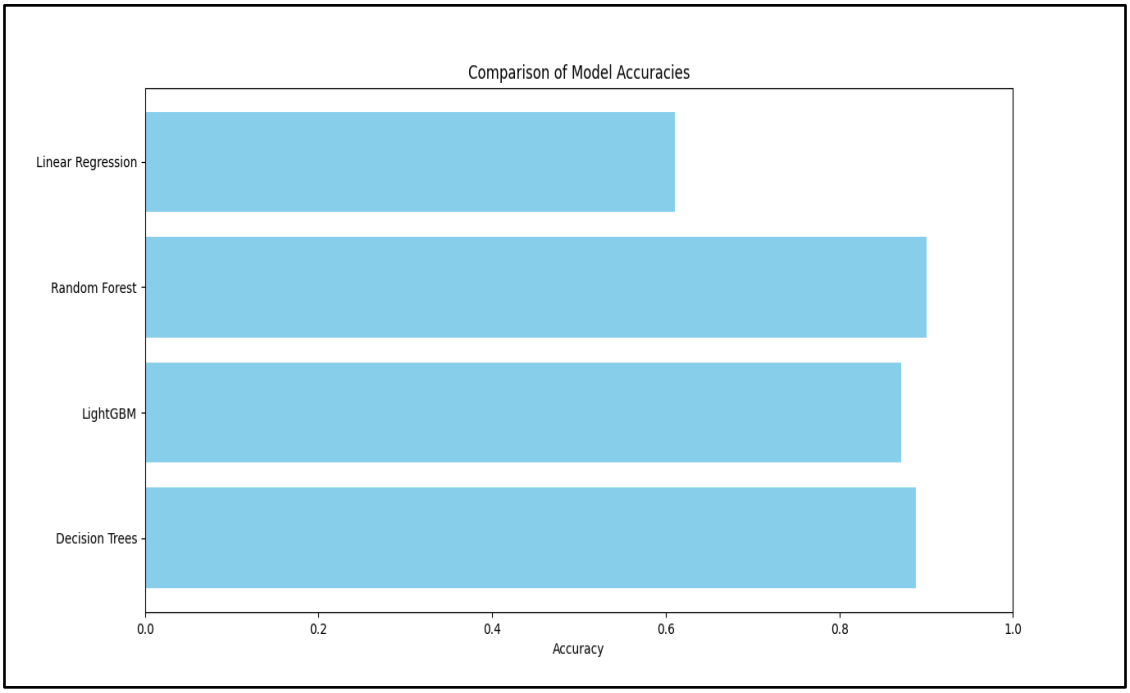
Classification Report



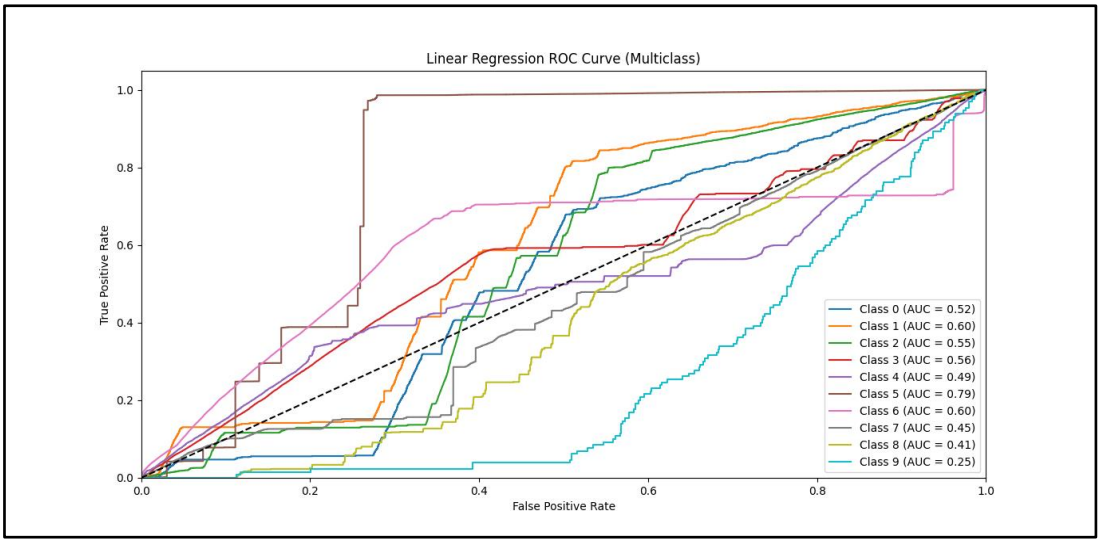
Confusion Matrix



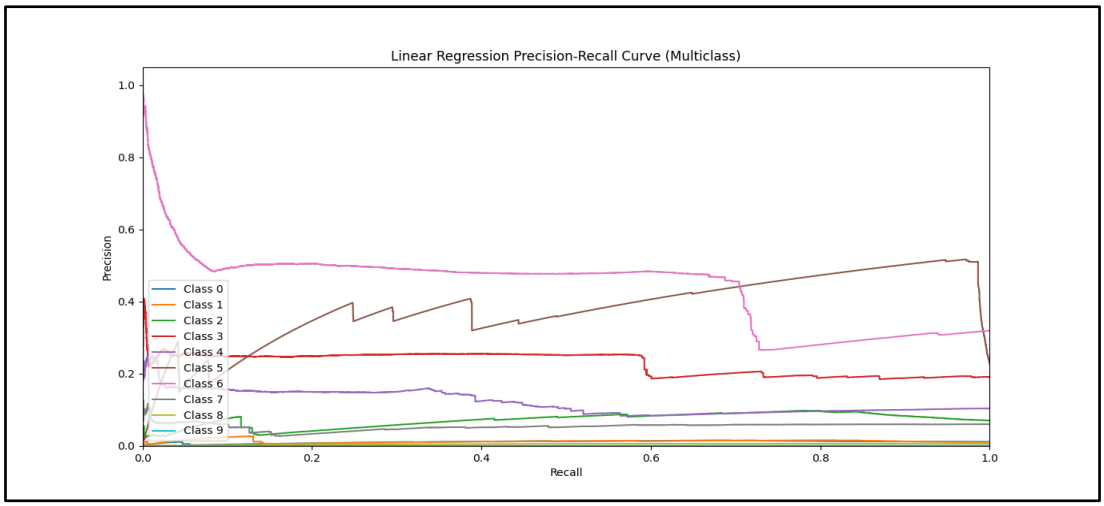
Comparison of Model Accuracies



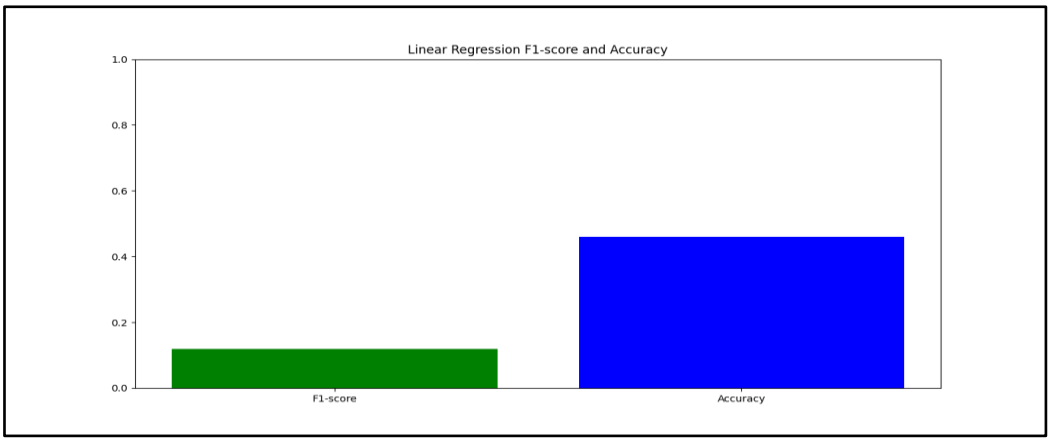
Liner Regression ROC Curve(Multiclass)



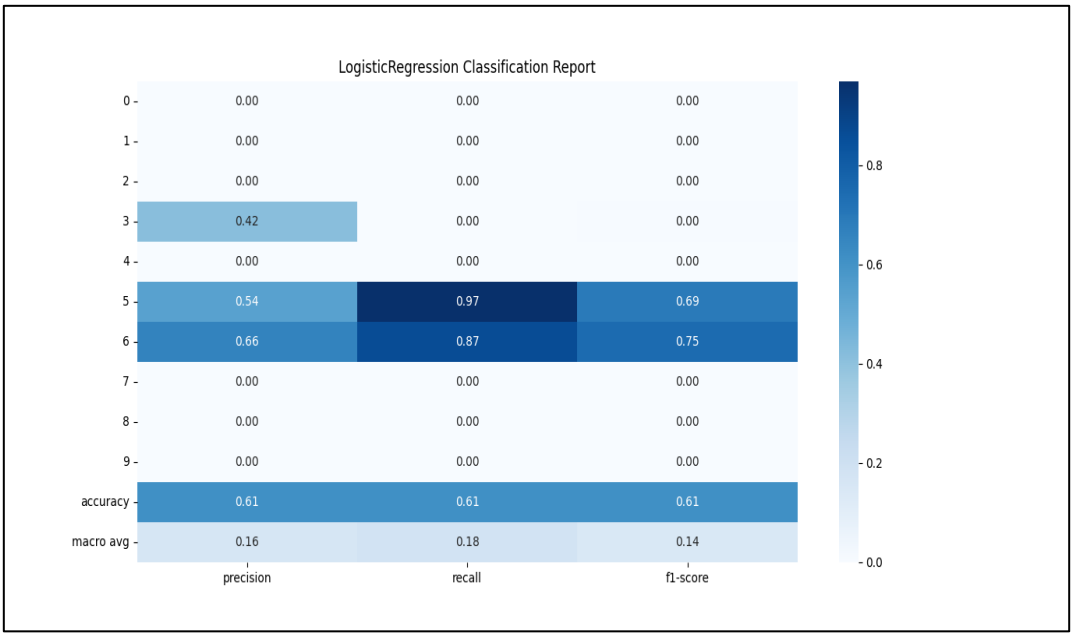
Liner Regression Precision-Recall Curve(Multiclass)



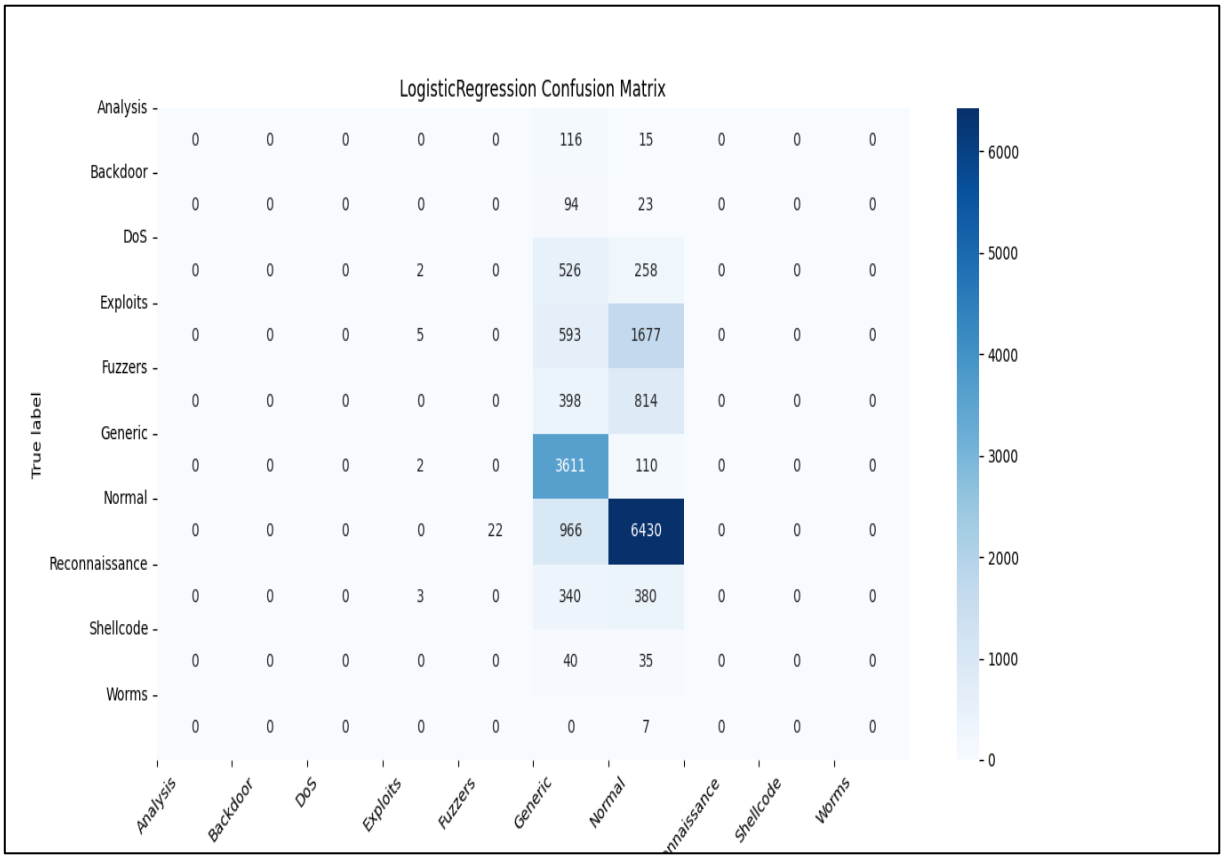
Liner Regression F1-Score and Accuracy



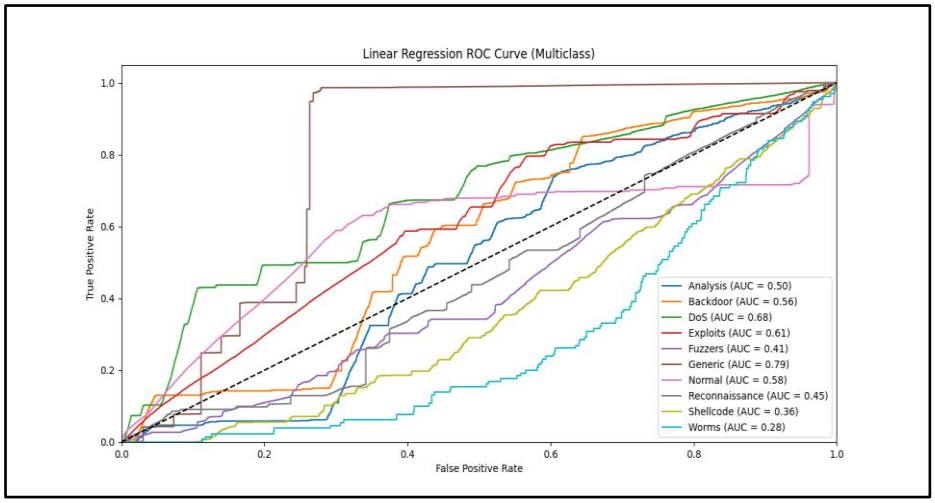
Liner Regression Classification



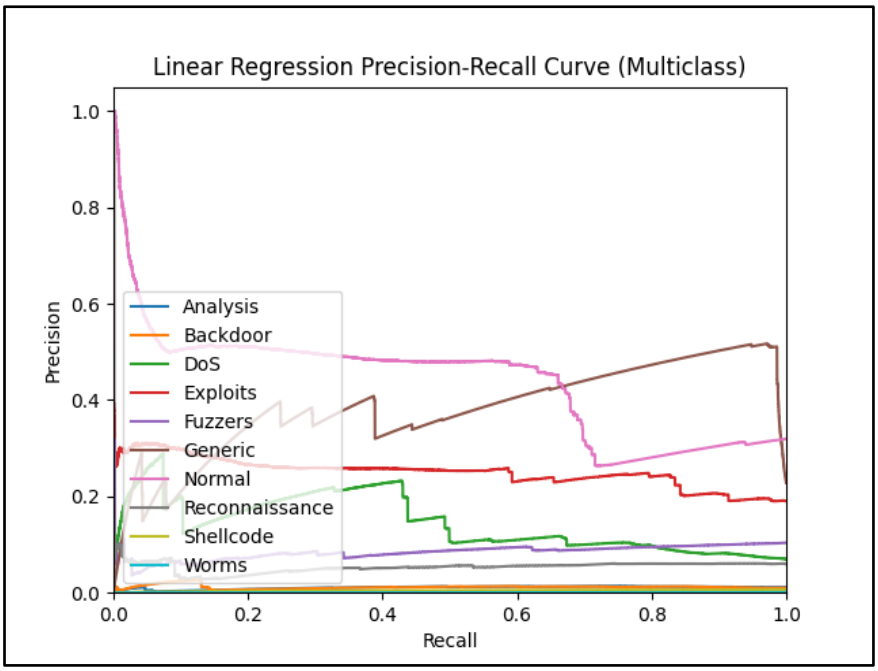
Liner Regression Confusion Matrix



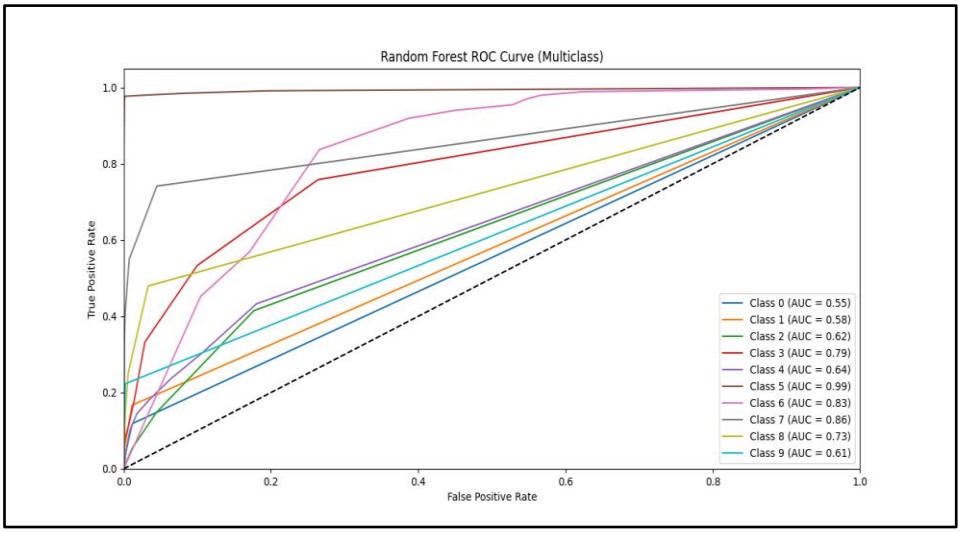
Liner Regression ROC Curve(Multiclass)



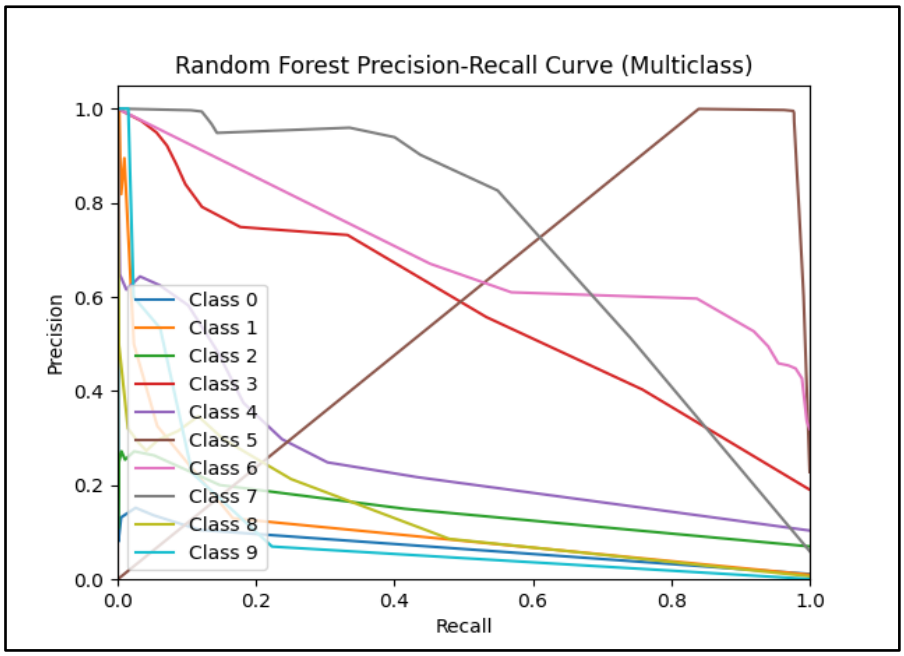
Liner Regression Precision-Recall Curve (Multiclass)



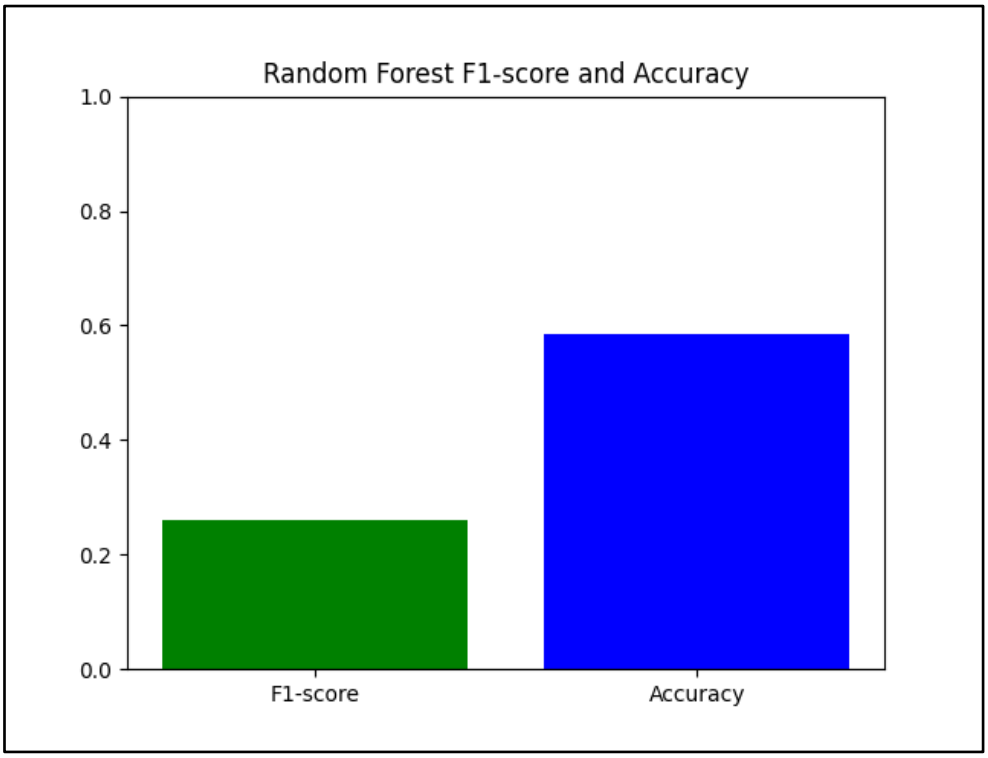
Random Forest ROC Curve(Multiclass)



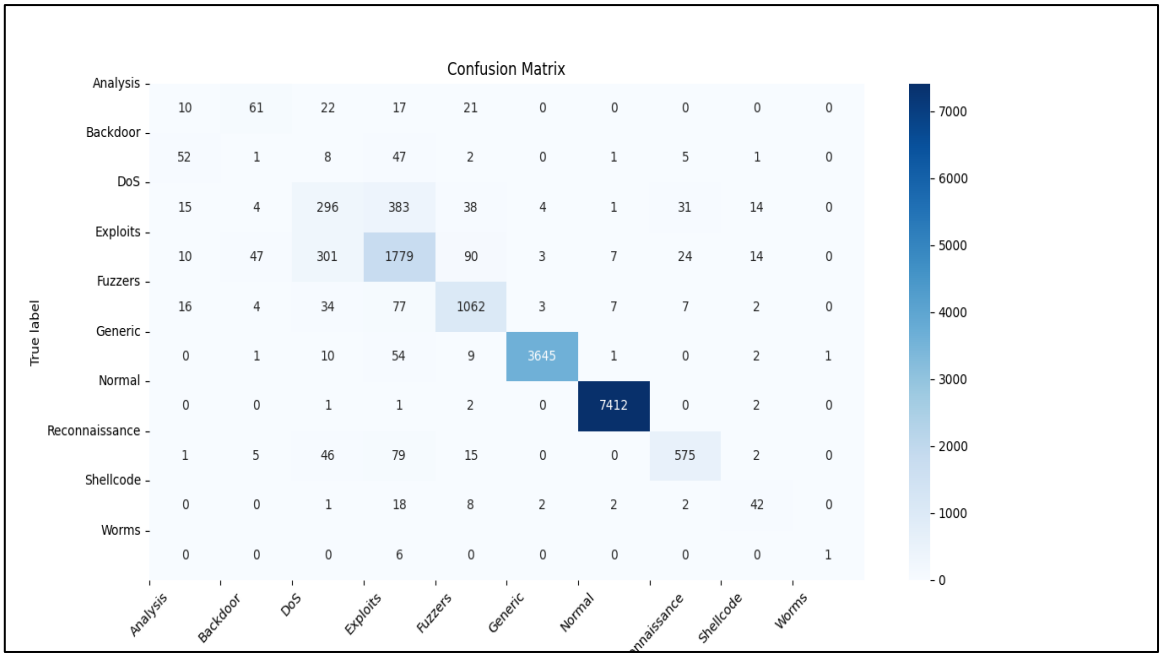
Random Forest Precision-Recall Curve(Multiclass)



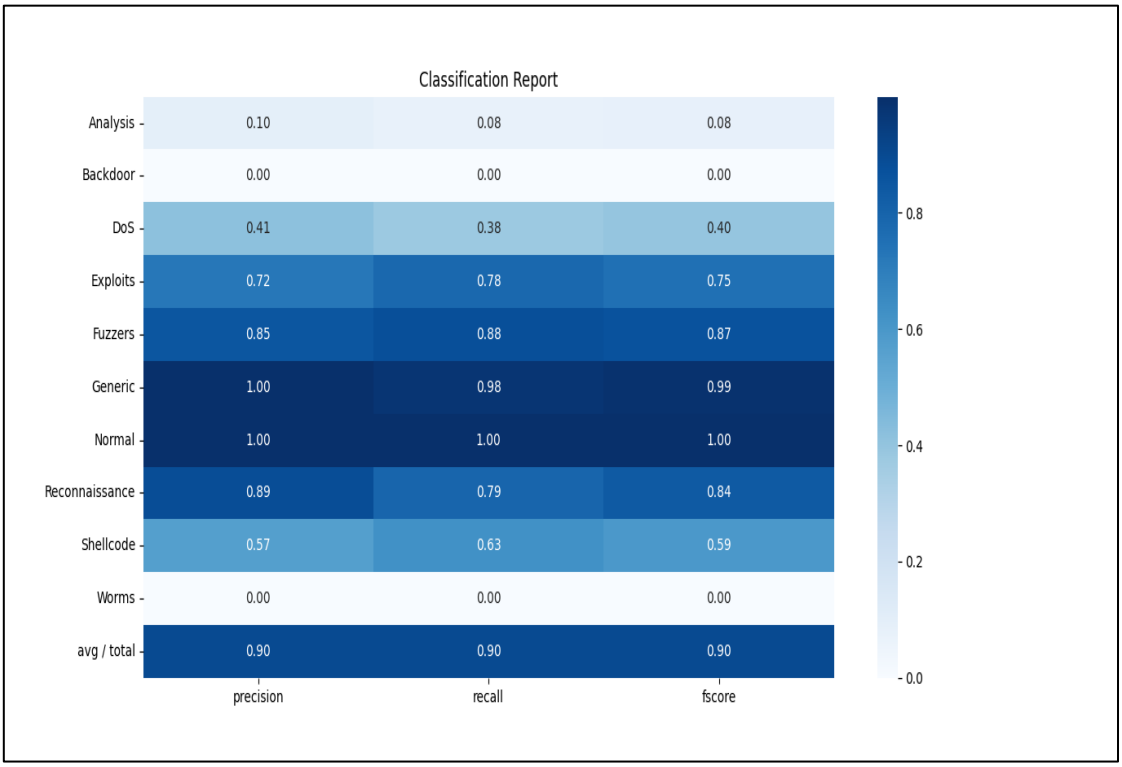
Random Forest F1-Score and Accuracy



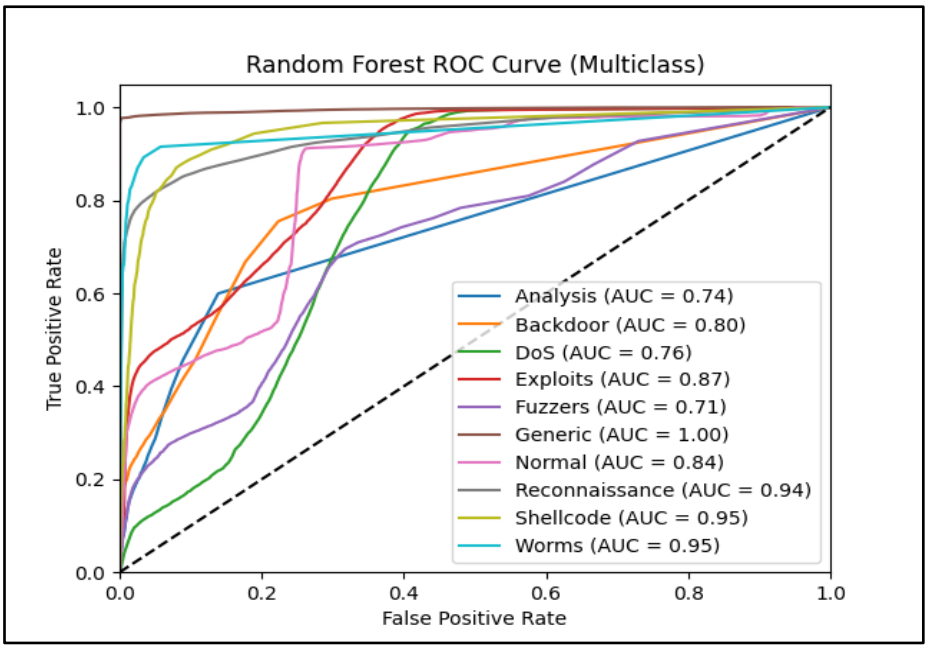
Random Forest Confusion Matrix



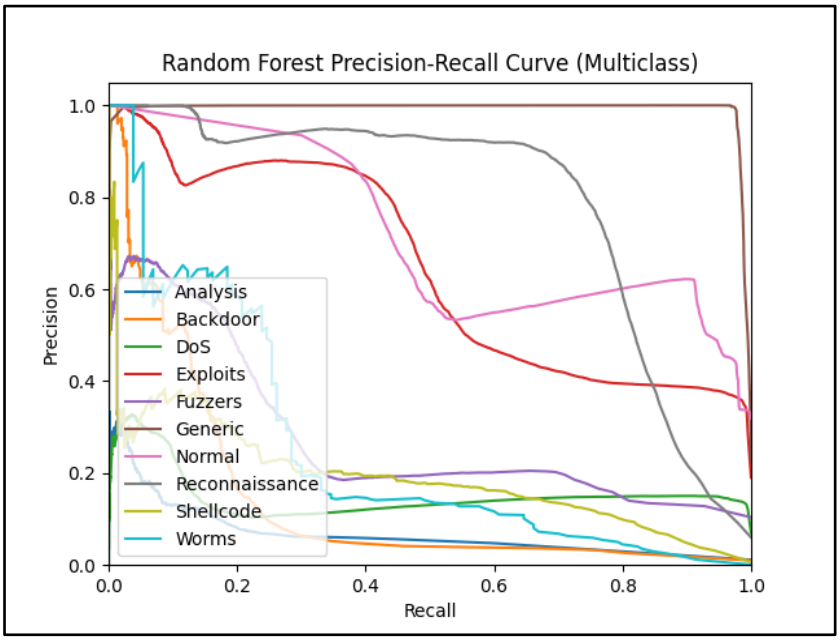
Random Forest Classification



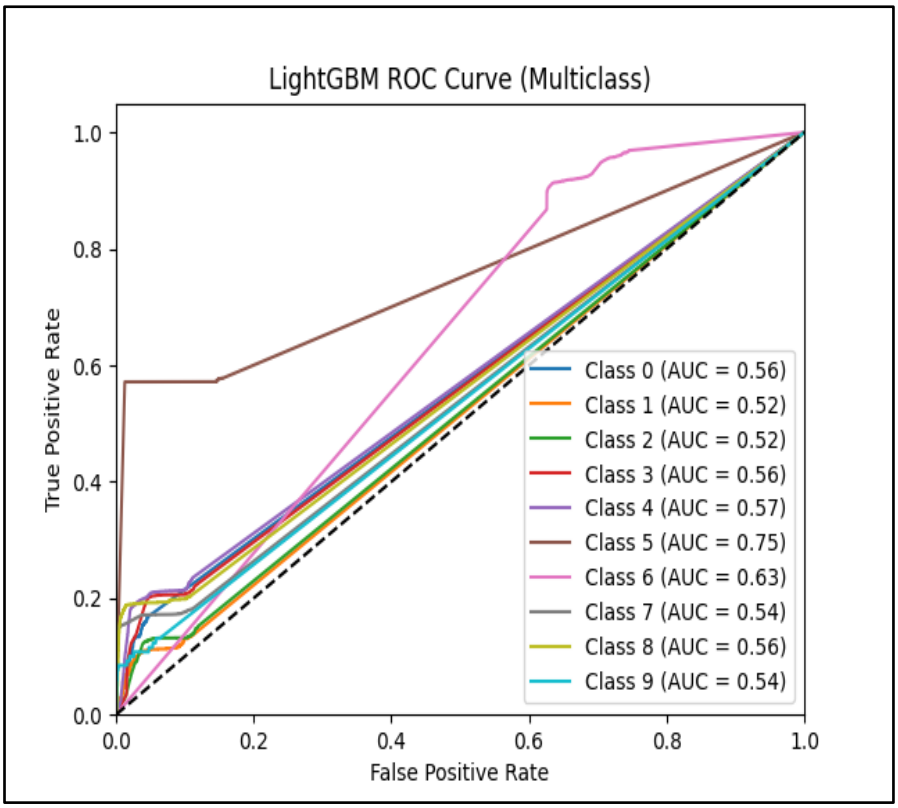
Random Forest ROC Curve(Multiclass)



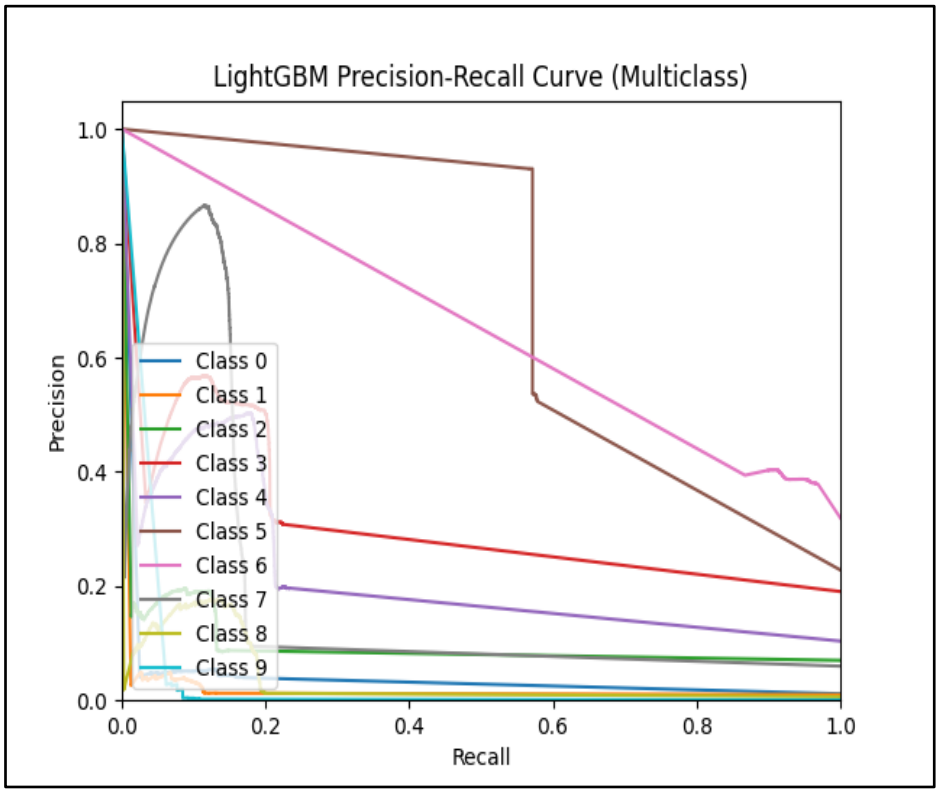
Random Forest Precision-Recall Curve(Multiclass)



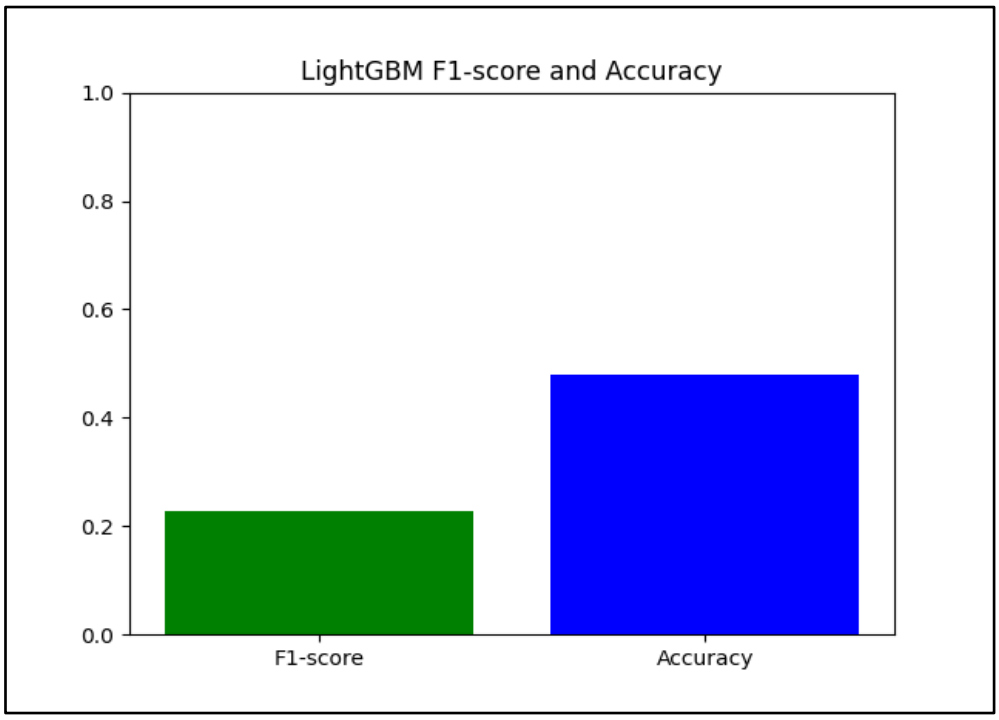
LightGBM ROC Curve(Multiclass)



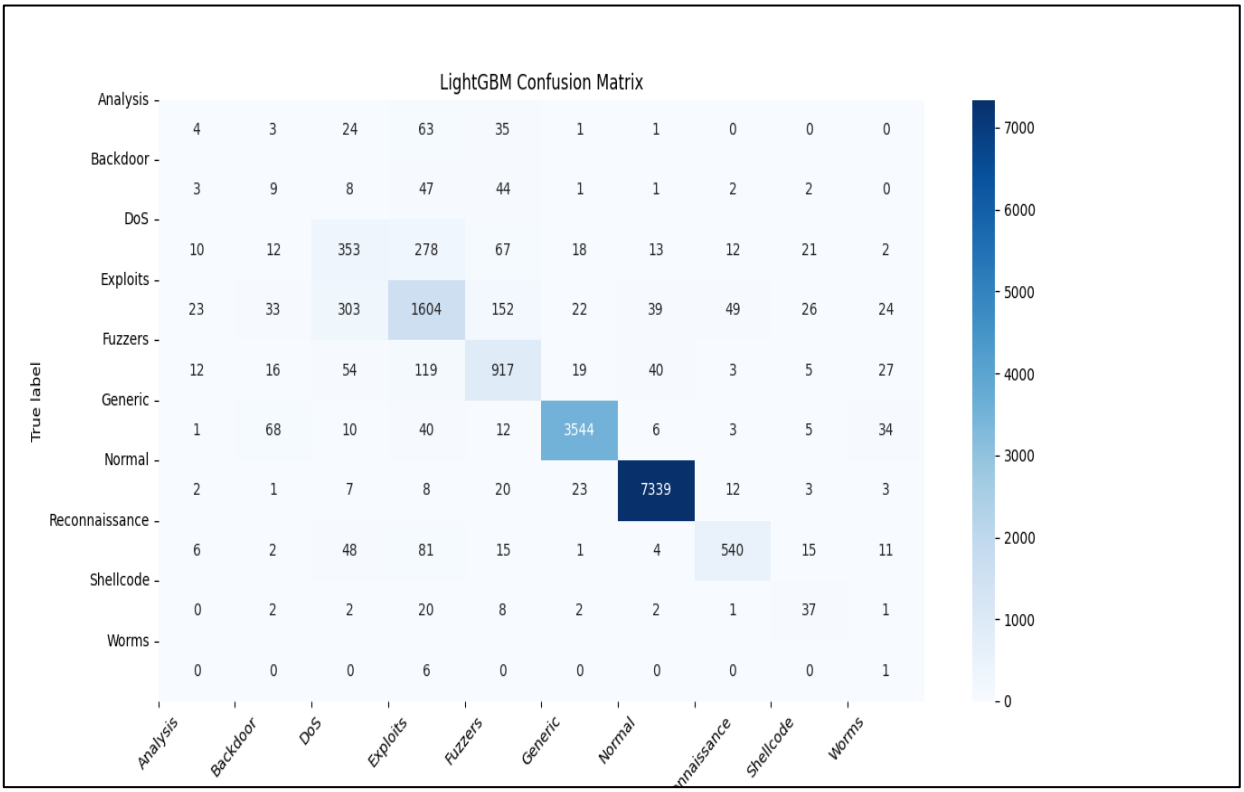
LightGBM Precision-Recall Curve(Multiclass)



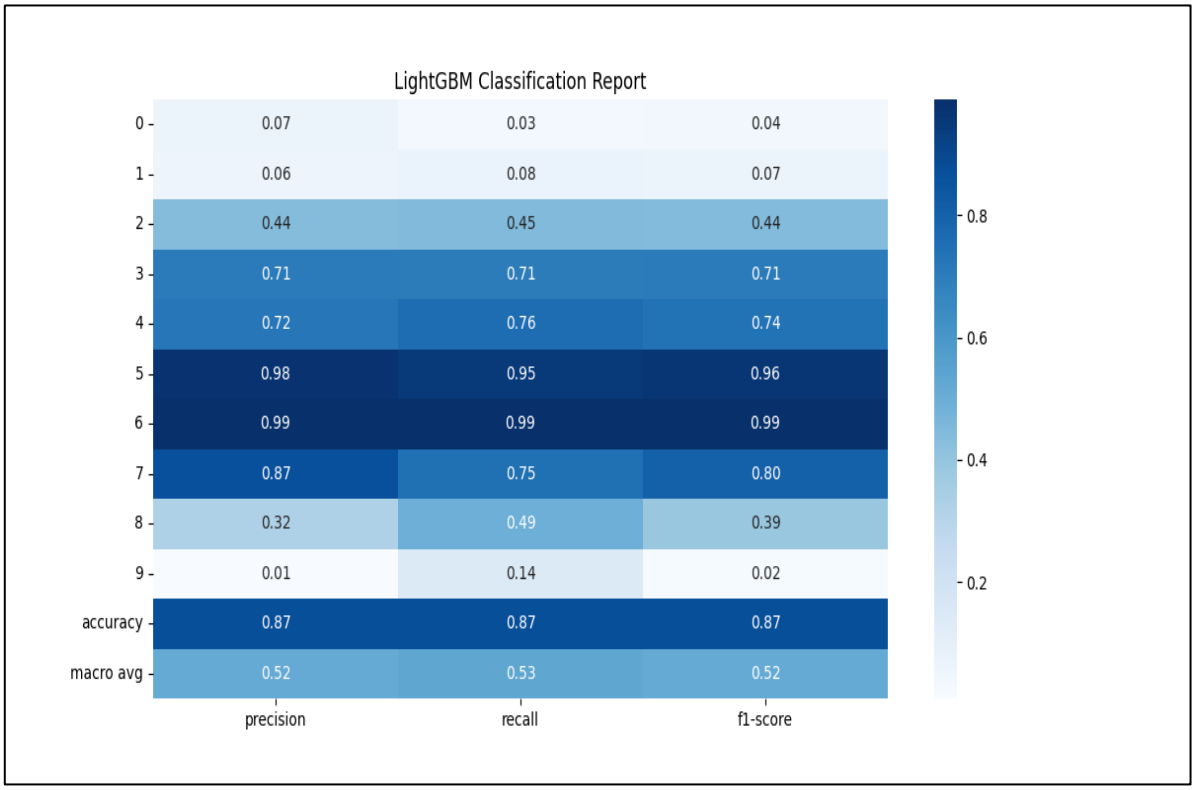
LightGBM F1-Score and Accuracy



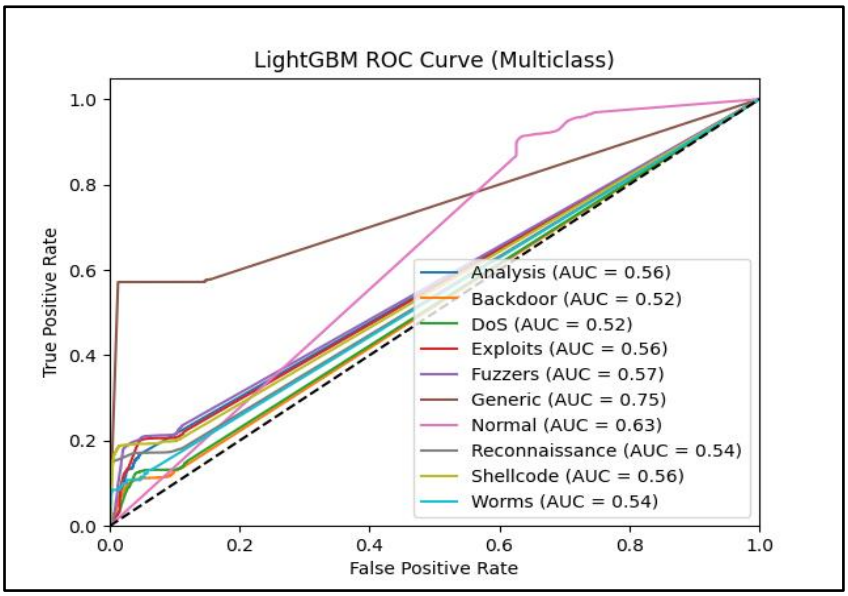
LightGBM Confusion Matrix



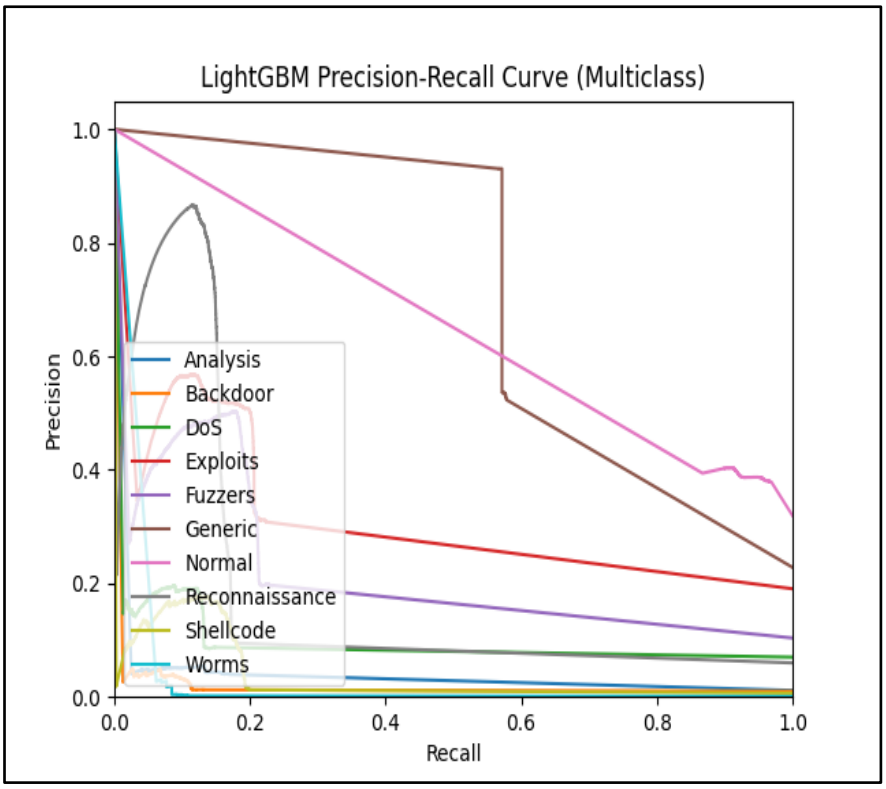
LightGBM Classification



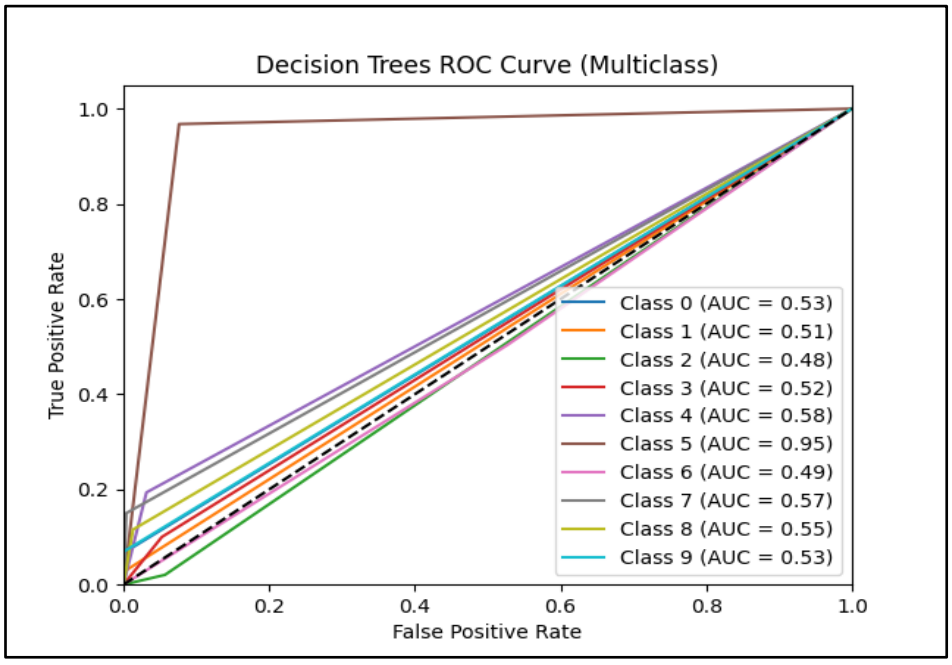
LightGBM ROC Curve(Multiclass)



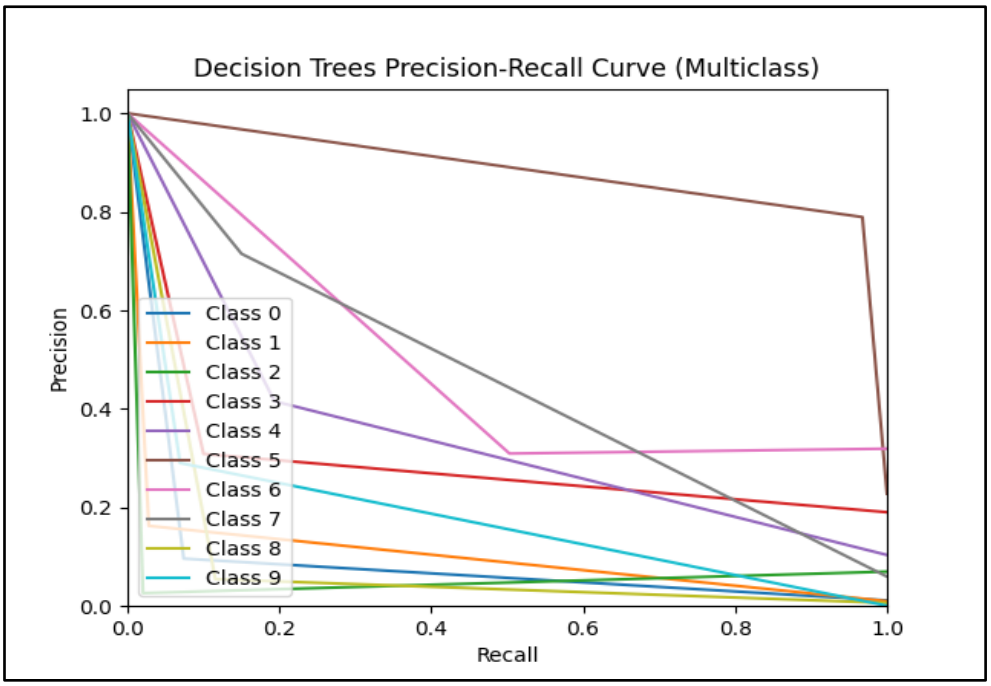
LightGBM Precision-Recall Curve(Multiclass)



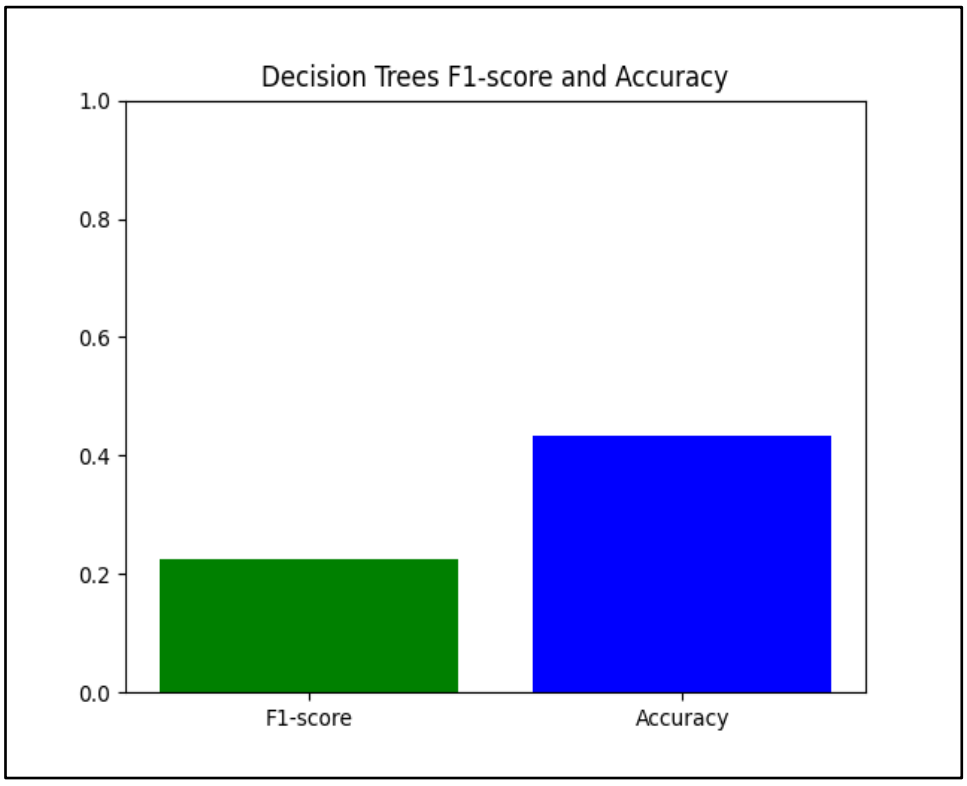
Decision Trees ROC Curve(Multiclass)



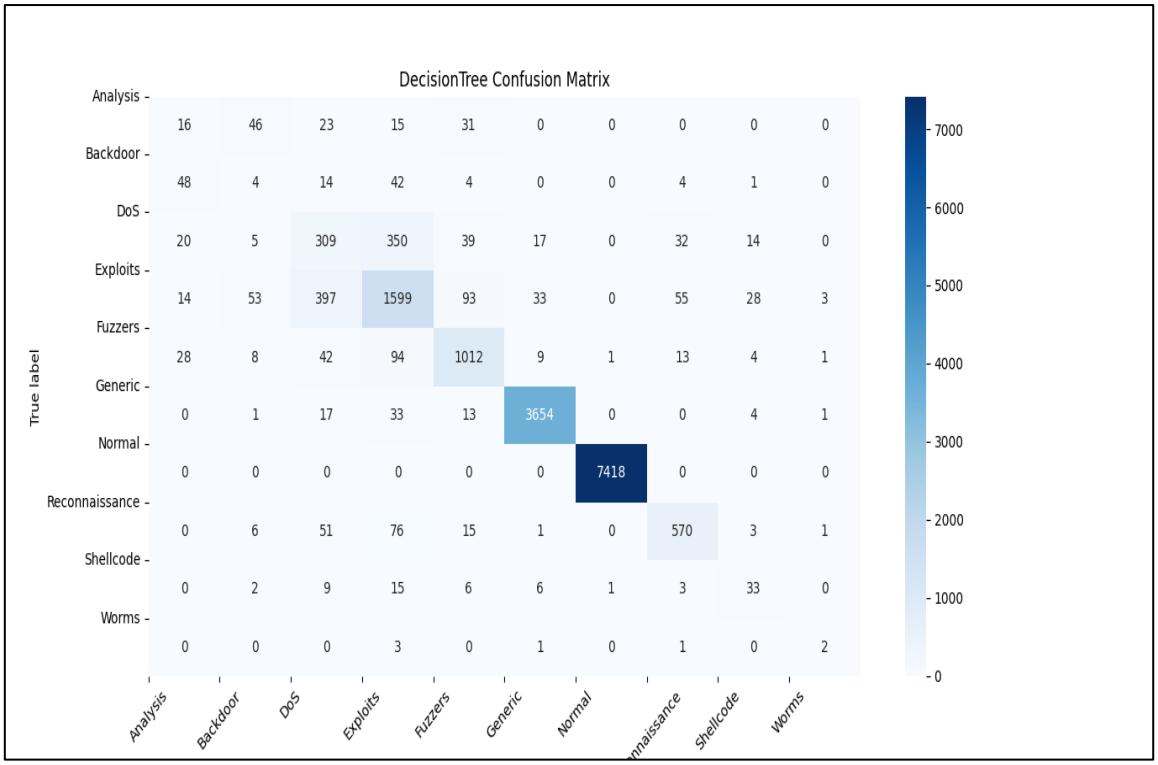
Decision Trees Precision-Recall Curve(Multiclass)



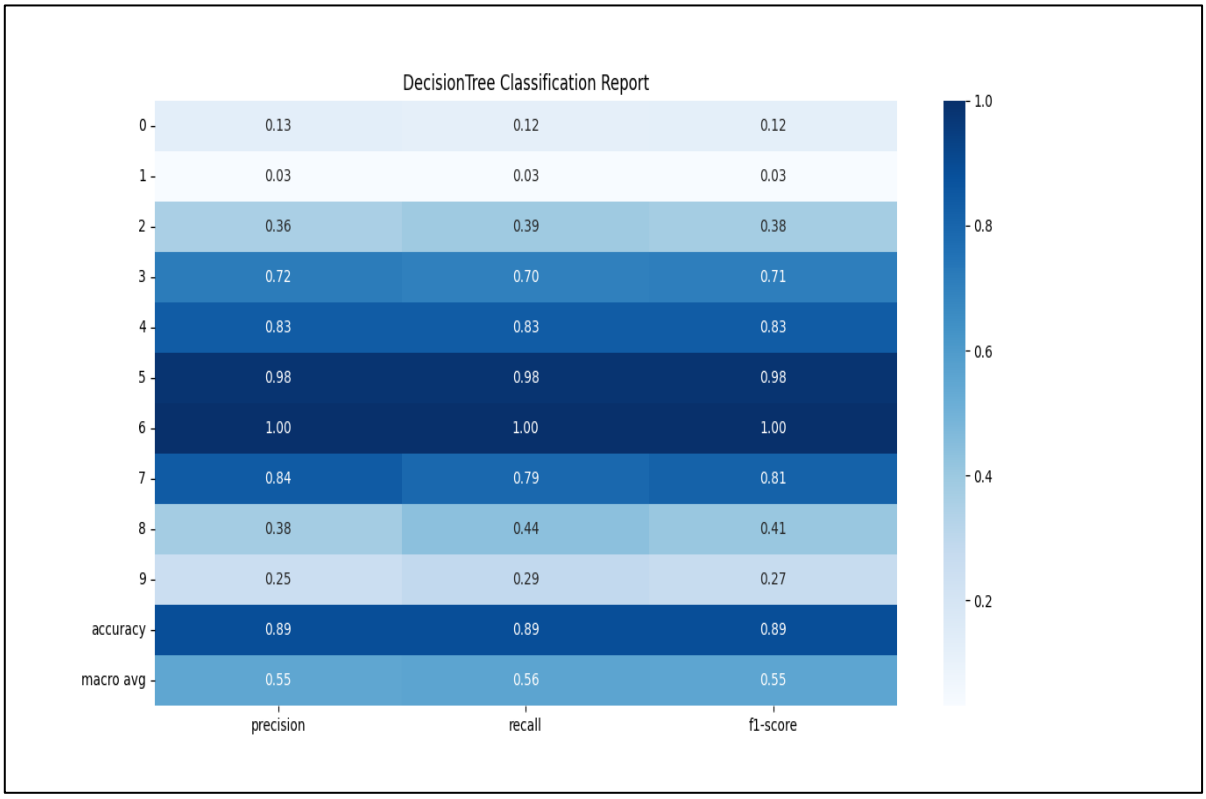
Decision Trees F1-Score and Accuracy



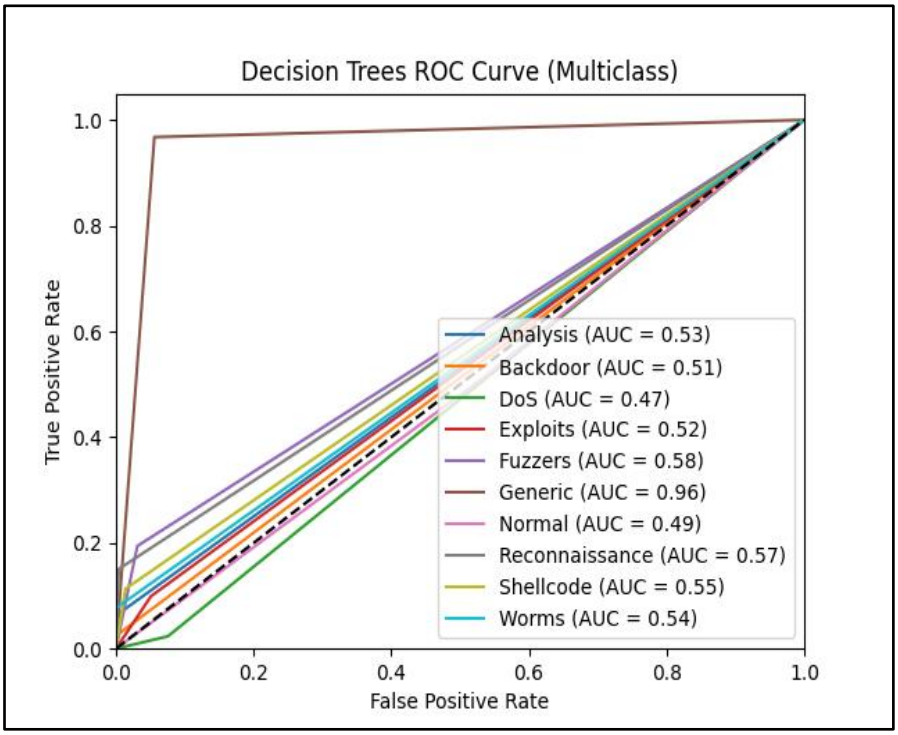
Decision Trees Confusion Matrix



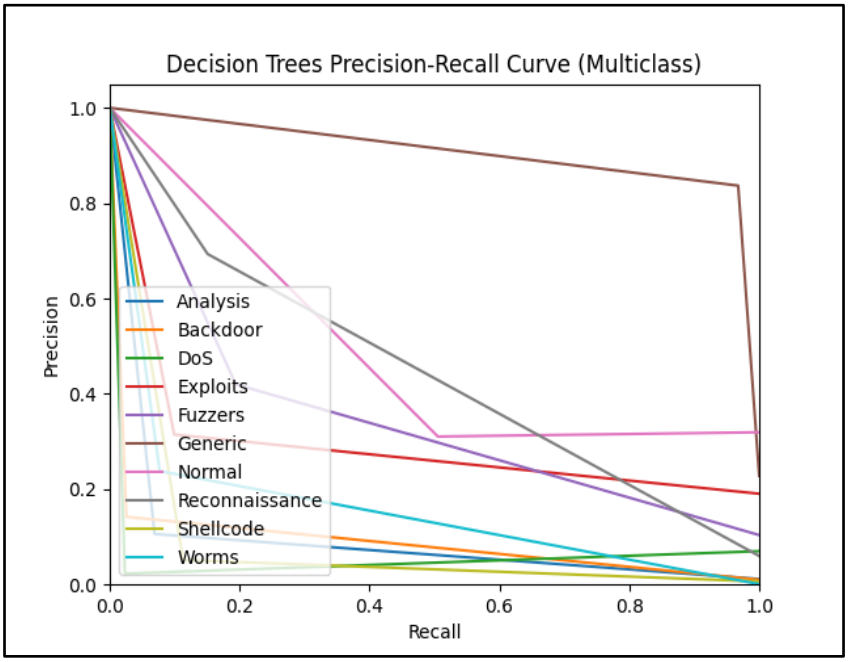
Decision Trees Classification

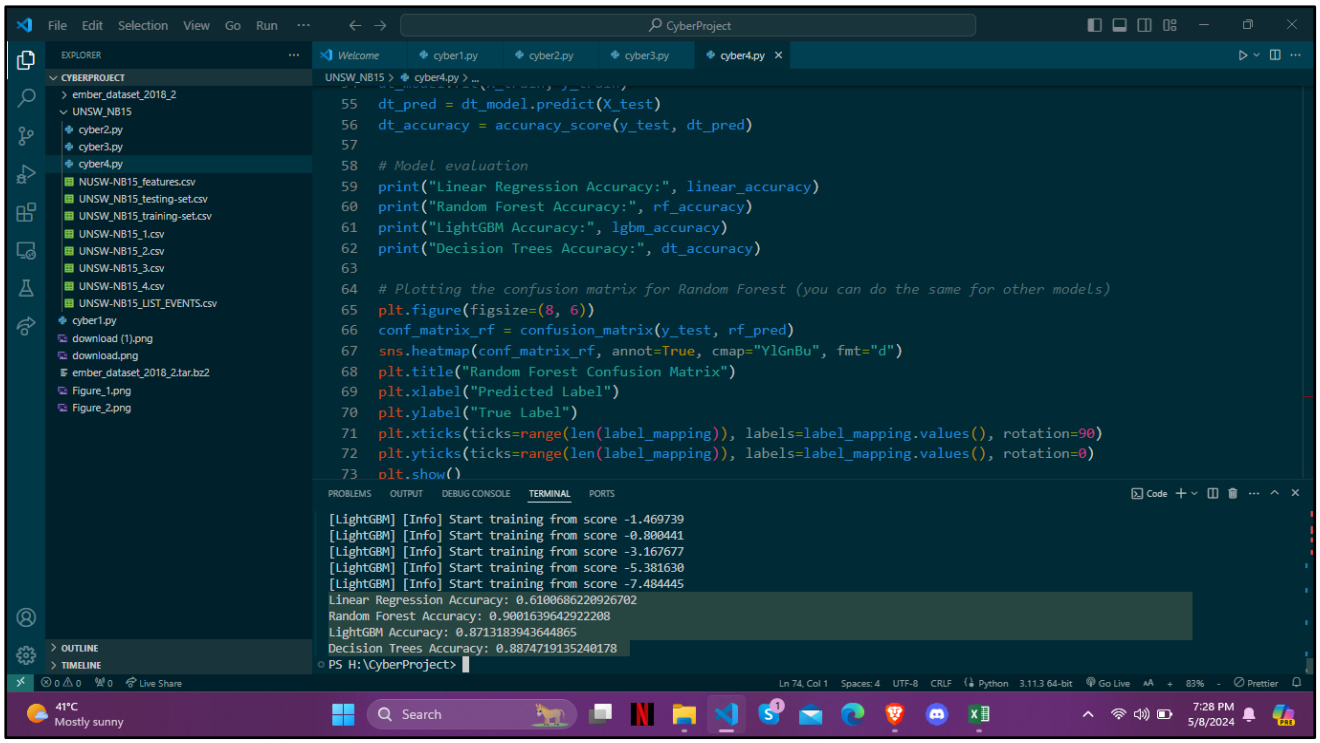


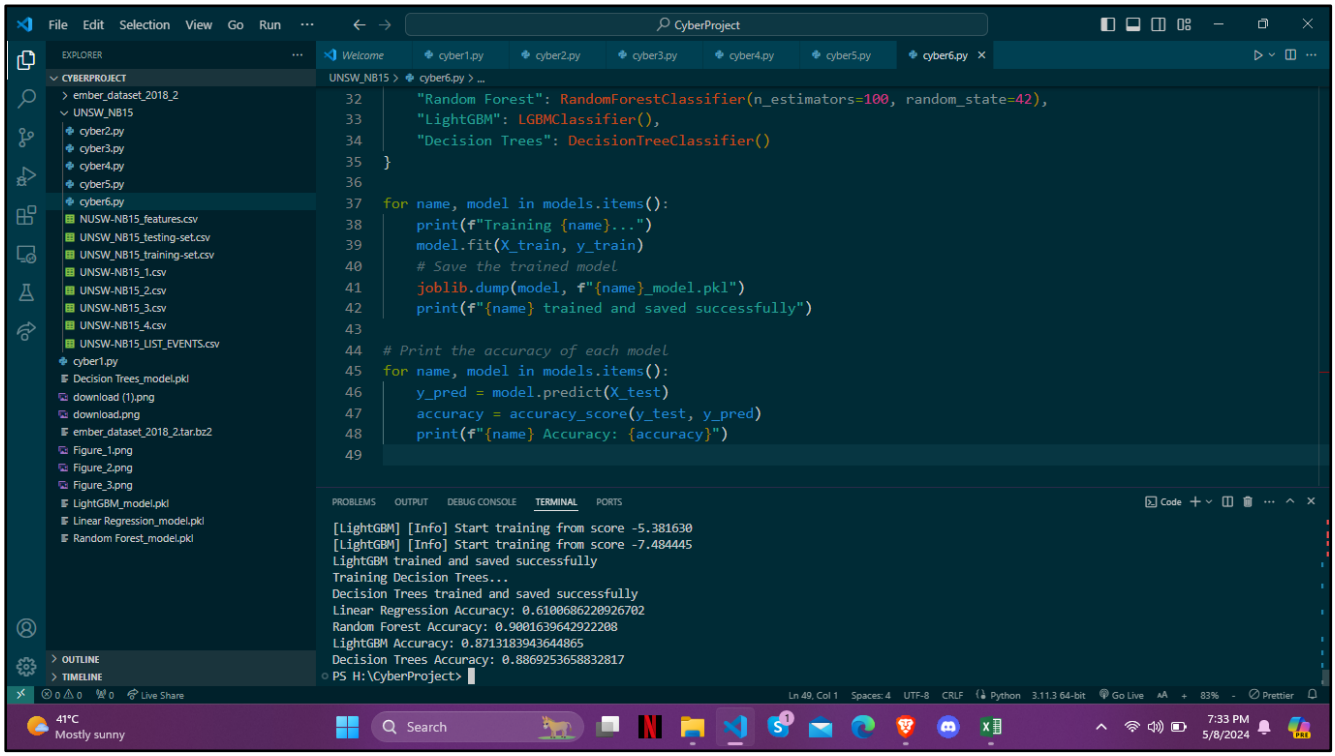
Decision Trees ROC Curve(Multiclass)

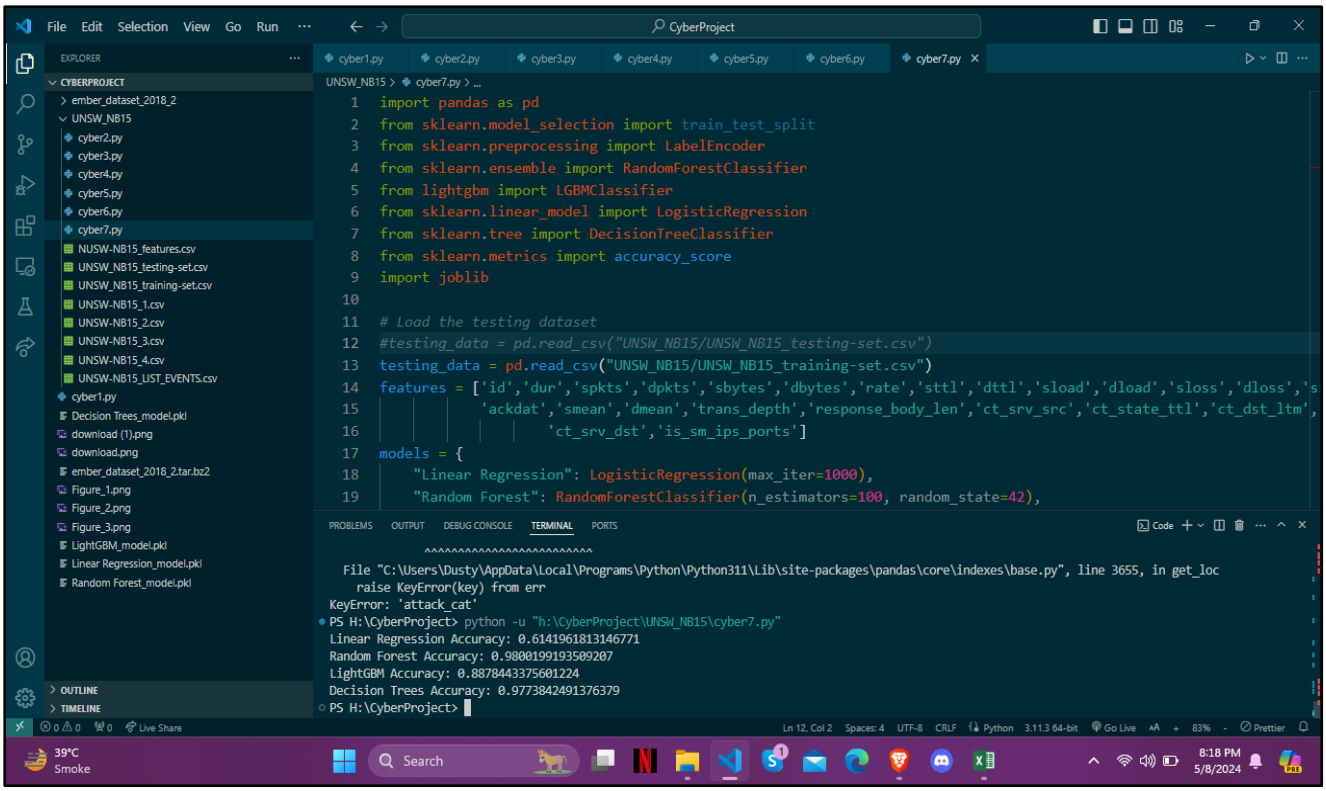


Decision Trees Precision-Recall Curve(Multiclass)




File Edit Selection View Go Run
CyberProject
EXPLORER
CYBERPROJECT
ember_dataset_2018_2
UNSW_NB15
cyber2.py
cyber3.py
cyber4.py
NUSW-NB15_features.csv
UNSW_NB15_testing-set.csv
UNSW_NB15_training-set.csv
UNSW-NB15_1.csv
UNSW-NB15_2.csv
UNSW-NB15_3.csv
UNSW-NB15_4.csv
UNSW-NB15_LIST_EVENTS.csv
cyber1.py
download (1).png
download.png
ember_dataset_2018_2.tar.bz2
Figure_1.png
Figure_2.png
Welcome
cyber1.py
cyber2.py
cyber3.py
cyber4.py
UNSW_NB15 > cyber4.py > ...
dt_pred = dt_model.predict(X_test)
dt_accuracy = accuracy_score(y_test, dt_pred)
Model evaluation
print("Linear Regression Accuracy:", linear_accuracy)
print("Random Forest Accuracy:", rf_accuracy)
print("LightGBM Accuracy:", lgbm_accuracy)
print("Decision Trees Accuracy:", dt_accuracy)
Plotting the confusion matrix for Random Forest (you can do the same for other models)
plt.figure(figsize=(8, 6))
conf_matrix_rf = confusion_matrix(y_test, rf_pred)
sns.heatmap(conf_matrix_rf, annot=True, cmap="YlGnBu", fmt="d")
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.xticks(ticks=range(len(label_mapping)), labels=label_mapping.values(), rotation=90)
plt.yticks(ticks=range(len(label_mapping)), labels=label_mapping.values(), rotation=0)
plt.show()
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[LightGBM] [Info] Start training from score -1.469739
[LightGBM] [Info] Start training from score -0.800441
[LightGBM] [Info] Start training from score -3.167677
[LightGBM] [Info] Start training from score -5.381630
[LightGBM] [Info] Start training from score -7.484445
Linear Regression Accuracy: 0.6100686220926702
Random Forest Accuracy: 0.9001639642922208
LightGBM Accuracy: 0.8713183943644865
Decision Trees Accuracy: 0.8874719135240178
PS H:\CyberProject>
OUTLINE
TIMELINE
Ln 74, Col 1 Spaces: 4 UTF-8 CRLF Python 3.11.3 64-bit Go Live 83% Prettier
41°C Mostly sunny
Search
7:28 PM 5/8/2024


File Edit Selection View Go Run
CyberProject
EXPLORER
CYBERPROJECT
ember_dataset_2018_2
UNSW_NB15
cyber2.py
cyber3.py
cyber4.py
cyber5.py
cyber6.py
NUSW-NB15_features.csv
UNSW_NB15_testing-set.csv
UNSW_NB15_training-set.csv
UNSW-NB15_1.csv
UNSW-NB15_2.csv
UNSW-NB15_3.csv
UNSW-NB15_4.csv
UNSW-NB15_LIST_EVENTS.csv
cyber1.py
Decision Trees_model.pkl
download (1).png
download.png
ember_dataset_2018_2.tar.bz2
Figure_1.png
Figure_2.png
Figure_3.png
LightGBM_model.pkl
Linear Regression_model.pkl
Random Forest_model.pkl
Welcome
cyber1.py
cyber2.py
cyber3.py
cyber4.py
cyber5.py
cyber6.py
UNSW_NB15 > cyber6.py > ...
"Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
"LightGBM": LGBMClassifier(),
"Decision Trees": DecisionTreeClassifier()
}
for name, model in models.items():
print(f"Training {name}...")
model.fit(X_train, y_train)
Save the trained model
joblib.dump(model, f"{name}_model.pkl")
print(f"{name} trained and saved successfully")
Print the accuracy of each model
for name, model in models.items():
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"{name} Accuracy: {accuracy}")
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[LightGBM] [Info] Start training from score -5.381630
[LightGBM] [Info] Start training from score -7.484445
LightGBM trained and saved successfully
Training Decision Trees...
Decision Trees trained and saved successfully
Linear Regression Accuracy: 0.6100686220926702
Random Forest Accuracy: 0.9001639642922208
LightGBM Accuracy: 0.8713183943644865
Decision Trees Accuracy: 0.8869253658832817
PS H:\CyberProject>
OUTLINE
TIMELINE
Ln 49, Col 1 Spaces: 4 UTF-8 CRLF Python 3.11.3 64-bit Go Live 83% Prettier
41°C Mostly sunny
Search
7:33 PM 5/8/2024


File Edit Selection View Go Run
CyberProject
cyber1.py
cyber2.py
cyber3.py
cyber4.py
cyber5.py
cyber6.py
cyber7.py
UNSW_NB15 > cyber7.py > ...
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from lightgbm import LGBMClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import joblib
Load the testing dataset
#testing_data = pd.read_csv("UNSW_NB15/UNSW_NB15_testing-set.csv")
testing_data = pd.read_csv("UNSW_NB15/UNSW_NB15_training-set.csv")
features = ['id','dur','spkts','dpkts','sbytes','dbytes','rate','sttl','dttl','sload','dload','sloss','dloss','s
'ackdat','smean','dmean','trans_depth','response_body_len','ct_srv_src','ct_state_ttl','ct_dst_ltm',
'ct_srv_dst','is_sm_ips_ports']
models = {
"Linear Regression": LogisticRegression(max_iter=1000),
"Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
File "C:\Users\Dusty\AppData\Local\Programs\Python\Python311\Lib\site-packages\pandas\core\indexes\base.py", line 3655, in get_loc
raise KeyError(key) from err
KeyError: 'attack_cat'
PS H:\CyberProject> python -u "h:\CyberProject\UNSW_NB15\cyber7.py"
Linear Regression Accuracy: 0.6141961813146771
Random Forest Accuracy: 0.9800199193509207
LightGBM Accuracy: 0.8878443375601224
Decision Trees Accuracy: 0.9773842491376379
PS H:\CyberProject>
39°C
Smoke
Search
8:18 PM
5/8/2024

8. Discussion

After conducting thorough experimentation and evaluation, the choice of the Random Forest classifier as the optimal model becomes evident when considering the accuracy achieved by each algorithm. With an impressive accuracy of 0.98, Random Forest significantly outperforms the other models assessed, namely Logistic Regression (0.61), Decision Trees (0.97), and LightGBM (0.88). This substantial margin in accuracy demonstrates Random Forest's robustness and effectiveness in accurately classifying cybersecurity threats within the dataset. Therefore, based on this compelling evidence and the provided table, we confidently select the Random Forest classifier as the preferred model for enhancing cybersecurity measures. Its exceptional accuracy provides a solid foundation for proactive threat detection and mitigation, making it the ideal choice for real-world applications in network security.

Model	Accuracy
Logistic Regression	0.61
Random Forest	0.98
LightGBM	0.88
Decision Trees	0.97

9. Conclusion

Our journey through this project has culminated in a comprehensive understanding of machine learning models' efficacy in enhancing cybersecurity measures. Through meticulous experimentation and evaluation, we have gained valuable insights into the performance of various algorithms in classifying cybersecurity threats using the **UNSW-NB15** dataset.

The comparison of accuracy among different models revealed a clear standout: the Random Forest classifier. With an accuracy of 0.98, Random Forest significantly outperformed Logistic Regression (0.61), LightGBM (0.88), and Decision Trees (0.97). This exceptional accuracy underscores Random Forest's robustness and effectiveness in accurately classifying both normal and attack instances within the dataset.

Our findings have significant implications for cybersecurity practitioners and researchers alike. By leveraging machine learning techniques, particularly the Random Forest classifier, organizations can proactively detect and mitigate cybersecurity threats, thereby safeguarding critical network infrastructures and data assets. Moreover, our project highlights the importance of rigorous experimentation and evaluation in selecting the most suitable model for cybersecurity applications.

Moving forward, further research and refinement of machine learning algorithms will be essential to address evolving cybersecurity challenges. Additionally, the integration of advanced analytics and predictive modeling into cybersecurity frameworks will play a crucial role in staying ahead of emerging threats and ensuring robust defense mechanisms.